PathWise

Requirements Specification Document

March 27th, 2026.

	Student ID	Student Name & Surname
#1	150122050	Ömer Ercan DAYAN
#2	150123025	Furkan Eren GÜLÇAY
#3	150123028	Mustafa Haktan YAKUT

Prepared for

CSE3044 Software Engineering Term Project

TABLE OF CONTENTS

1. INTRODUCTION 10
1.1. PURPOSE 10
1.2. SCOPE 10
1.2.1. SYSTEM CAPABILITIES 10
1.2.2. SYSTEM OBJECTIVES AND GOALS 11
1.2.3. SYSTEM CONSTRAINTS AND LIMITATIONS 11
1.3. DEFINITIONS, ACRONYMS, AND ABBREVIATIONS 11
1.4. REFERENCES 12
1.5. OVERVIEW 12
2. GENERAL DESCRIPTION 14
2.1. PRODUCT PERSPECTIVE 14
2.2. PRODUCT FUNCTIONS 14
2.3. USER CHARACTERISTICS 15
2.4. GENERAL CONSTRAINTS 15
2.5. ASSUMPTIONS AND DEPENDENCIES 15
3. SPECIFIC REQUIREMENTS 17
3.1. EXTERNAL INTERFACE REQUIREMENTS 17
3.1.1. USER INTERFACES 17
3.1.1.1. LOGIN PAGES 17
3.1.1.2. REGISTRATION PAGE 18
3.1.1.3. ADMIN PANEL PAGE 18
3.1.1.4. USER DASHBOARD 19
3.1.1.5. OWNER DASHBOARD 19
3.1.1.6. RECOMMENDATION PAGE 20
3.1.1.7. BUSINESS VIEW PAGE 21
3.1.1.8. VENUE ADDING PAGE 21
3.1.1.9. VENUE SEARCH PAGE 22
3.1.2. HARDWARE INTERFACES 22
3.1.3. SOFTWARE INTERFACES 23
3.1.3.1. FRONTEND INTERFACES 23
3.1.3.2. BACKEND INTERFACES 23
3.1.3.3. DATABASE INTERFACES 23
3.1.4. COMMUNICATION INTERFACE 23

3.2. FUNCTIONAL REQUIREMENTS 25
3.2.1. FR #1: USER AUTHENTICATION AND AUTHORIZATION 25
3.2.1.1. DESCRIPTION 25
3.2.1.2. INPUTS 25
3.2.1.3. PROCESSING 25
3.2.1.4. OUTPUTS 25
3.2.1.5. CONSTRAINTS 26
3.2.1.6. ERROR HANDLING 26
3.2.2. FR #2: TRAVEL PLAN GENERATION 26
3.2.2.1. DESCRIPTION 26
3.2.2.2. INPUTS 27
3.2.2.3. PROCESSING 27
3.2.2.4. OUTPUTS 27
3.2.2.5. CONSTRAINTS 27
3.2.2.6. ERROR HANDLING 28
3.2.3. FR #3: VENUE MANAGEMENT BY BUSINESS OWNERS 28
3.2.3.1. DESCRIPTION 28
3.2.3.2. INPUTS 28
3.2.3.3. PROCESSING 28
3.2.3.4. OUTPUTS 28
3.2.3.5. CONSTRAINTS 29
3.2.3.6. ERROR HANDLING 29
3.2.4. FR #4: ALTERNATIVE VENUE RECOMMENDATION 29
3.2.4.1. DESCRIPTION 29
3.2.4.2. INPUTS 29
3.2.4.3. PROCESSING 29
3.2.4.4. OUTPUTS 30
3.2.4.5. CONSTRAINTS 30
3.2.4.6. ERROR HANDLING 30
3.2.5. FR #5: ADMINISTRATIVE CONTENT MODERATION 30
3.2.5.1. DESCRIPTION 30
3.2.5.2. INPUTS 30
3.2.5.3. PROCESSING 31
3.2.5.4. OUTPUTS 31
3.2.5.5. CONSTRAINTS 31
3.2.5.6. ERROR HANDLING 31

3.2.6. FR #6: USER ACCOUNT DELETION 31
3.2.6.1. DESCRIPTION 31
3.2.6.2. INPUTS 31
3.2.6.3. PROCESSING 32
3.2.6.4. OUTPUTS 32
3.2.6.5. CONSTRAINTS 32
3.2.6.6. ERROR HANDLING 32
3.2.7. FR #7: PREFERENCE PROFILE RESET 32
3.2.7.1. DESCRIPTION 32
3.2.7.2. INPUTS 32
3.2.7.3. PROCESSING 33
3.2.7.4. OUTPUTS 33
3.2.7.5. CONSTRAINTS 33
3.2.7.6. ERROR HANDLING 33
3.3. NON-FUNCTIONAL REQUIREMENTS 34
3.3.1. PERFORMANCE 34
3.3.2. RELIABILITY 34
3.3.3. AVAILABILITY 34
3.3.4. SECURITY 35
3.3.5. MAINTAINABILITY 35
3.3.6. PORTABILITY 35
3.4. INVERSE REQUIREMENTS 37
3.5. DESIGN CONSTRAINTS 38
3.5.1. TECHNOLOGY STACK CONSTRAINTS 38
3.5.2. WEB-BASED ARCHITECTURE 38
3.5.3. LOCAL DEVELOPMENT ENVIRONMENT 38
3.5.4. DATA INTEGRITY 38
3.5.5. SECURITY CONSTRAINTS 38
3.6. LOGICAL DATABASE REQUIREMENTS 40
The system requires a structured relational database to store and manage all application data. The database will be designed following relational database principles to ensure data consistency, integrity, and efficient querying. 40
The logical database structure is derived from the system's domain model and functional requirements. The main data entities include users, venues, recommendations, and related metadata such as tags. The database will maintain relationships between users, venues, and recommendation results generated by the system 40
3.6.1. User Data 40

The system must store information about registered users. Each user record contains basic account information and role identification. 40
The role attribute determines the permissions and system access level for each user. 40
3.6.2. Venue Data 40
The system must store information about venues submitted by business owners. 40
Attributes include: 40
- venueID (unique identifier) 40
- venueName 40
- location 40
- category 40
- tags 40
- ownerID (reference to the business owner) 40

Venues are associated with business owners and may contain descriptive tags that support the recommendation system 40
3.6.3. Recommendation Data 40
Each recommendation may contain multiple venues suggested by the system 41
3.6.4. Tag Data 41
Venues may have multiple associated tags 41
3.6.5. Data Relationships 41
4. UML DIAGRAMS 42
4.1. USE CASES 42
4.1.1. USE CASE #1 42
4.1.2. USE CASE #2 44
4.1.3. USE CASE #3 46
4.1.4. USE CASE #4 49
4.1.5. USE CASE #5 50
4.1.6. USE CASE #6 53
4.1.7. USE CASE #7 56
4.1.8. USE CASE #8 59
4.2. CLASSES / OBJECTS 63
Tracing: Records system activities, algorithm decision logs and error states to facilitate debugging and support the project's technical decision-tracking requirements. 63
4.3. SEQUENCE DIAGRAMS 64
5. APPENDICES 71
5.1. APPENDIX A: DECISION LOG 71

5.1.1. DECISION #1: TECHNOLOGY STACK SELECTION 71
5.1.1.1. DECISION MADE 71
5.1.1.2. ALTERNATIVES CONSIDERED 71
5.1.1.3. RATIONALE 71
5.1.1.4. TRADE-OFFS 72
5.1.1.5. RISKS OR TECHNICAL DEBTS 72
5.1.2. DECISION #2: RECOMMENDATION APPROACH 72
5.1.2.1. DECISION MADE 72
5.1.2.2. ALTERNATIVES CONSIDERED 72
5.1.2.3. RATIONALE 72
▪ The logic is simple, interpretable, and easy to implement 73
5.1.2.4. TRADE-OFFS 73
▪ Recommendation quality depends heavily on the accuracy and completeness of tags 73
5.1.2.5. RISKS OR TECHNICAL DEBTS 73
▪ Future improvements may require redesigning the recommendation logic 73
5.1.3. DECISION #3: RECOMMENDATION PLAN GENERATION 73
5.1.3.1. DECISION MADE 73
The system generates multiple alternative travel plans instead of a single recommendation. 73
5.1.3.2. ALTERNATIVES CONSIDERED 73
▪ Generating a single optimized travel plan 73
5.1.3.3. RATIONALE 73
▪ Providing multiple options allows users to compare and choose according to their preferences 73
▪ Improves user experience by offering flexibility 73
5.1.3.4. TRADE-OFFS 73
▪ Increased computational complexity 73
▪ More complex user interface design 73
5.1.3.5. RISKS OR TECHNICAL DEBTS 73
▪ Potential increase in response time 73
▪ Users may experience difficulty in decision-making if too many options are presented 73
5.1.4. DECISION #4: VENUE APPROVAL 73
5.1.4.1. DECISION MADE 73
5.1.4.2. ALTERNATIVES CONSIDERED 74
5.1.4.3. RATIONALE 74

- Ensures the accuracy and reliability of venue data 74
- Prevents invalid, incomplete, or misleading entries from being presented to users 74
- Maintains the overall quality and trustworthiness of the system 74

5.1.4.4. TRADE-OFFS 74

- Introduces delay between submission and visibility of venues 74
- Requires additional effort from administrators 74
- Reduces the level of system automation 74

5.1.4.5. RISKS OR TECHNICAL DEBTS 74

- Introduces delay between submission and visibility of venues 74
- Requires additional effort from administrators 74
- Reduces the level of system automation 74

5.1.5. DECISION #5: DEVELOPMENT ENVIRONMENT 74

5.1.5.1. DECISION MADE 74

The system operates in a local development environment and is not deployed to a production server. 74

5.1.5.2. ALTERNATIVES CONSIDERED 74

- Deployment to cloud-based platforms such as remote servers (These alternatives were not explored due to project scope and infrastructure limitations.) 74

5.1.5.3. RATIONALE 74

- Simplifies development, testing, and debugging processes 74
- Eliminates the need for infrastructure and deployment management 74
- Suitable for an academic project and demonstration purposes 74

5.1.5.4. TRADE-OFFS 75

- Limited accessibility, as the system is only available locally 75
- Lack of testing under real-world conditions such as network latency and high user load 75

5.1.5.5. RISKS OR TECHNICAL DEBTS 75

- System behavior may differ in a real production environment 75
- Future deployment may require additional configuration, security improvements, and performance optimization 75
- Integration with external systems may require further adjustments 75

5.1.6. DECISION #6: DATA STORAGE 75

5.1.6.1. DECISION MADE 75

A relational database management system is used for storing and managing system data. 75

5.1.6.2. ALTERNATIVES CONSIDERED 75

▪ Non-relational database systems (These alternatives were not explored in detail due to the structured nature of system data.) 75
5.1.6.3. RATIONALE 75
▪ The system contains structured data with clear relationships between entities such as users, venues, and recommendations 75
▪ Supports data integrity through constraints and relational structure 75
▪ Compatible with the object-relational mapping approach used in the system 75
5.1.6.4. TRADE-OFFS 75
▪ Less flexibility for handling unstructured data 75
▪ Schema modifications may require additional effort and database migration 75
5.1.6.5. RISKS OR TECHNICAL DEBTS 75
▪ Potential scalability limitations as the dataset grows 75
▪ Complex queries may impact performance if not optimized 75
▪ Future improvements may require query optimization or migration strategies 75
5.1.7. DECISION #7: RATING-BASED FEEDBACK MECHANISM 75
5.1.7.1. DECISION MADE 75
The system includes a simple rating-based feedback mechanism for evaluating recommendations. 76
5.1.7.2. ALTERNATIVES CONSIDERED 76
▪ No feedback mechanism 76
▪ Detailed review or comment system (More advanced feedback systems were not included due to scope limitations.) 76
5.1.7.3. RATIONALE 76
▪ Provides an easy and quick way for users to give feedback 76
▪ Helps evaluate the usefulness of generated recommendations 76
5.1.7.4. TRADE-OFFS 76
▪ Limited level of detail in user feedback 76
▪ Does not capture qualitative user opinions 76
5.1.7.5. RISKS OR TECHNICAL DEBTS 76
▪ Insufficient feedback data for improving recommendation logic 76
▪ May require future extension to more advanced feedback mechanisms 76
5.2. APPENDIX B: TEAM MEETINGS 77
..... 77
..... 78

Date	Document Version	Description	Author
27th March, 2026	v1.0	Complete and finalized version	PathWise project team

1. INTRODUCTION

This section defines the purpose and the scope of the PathWise Travel Recommendation System by outlining its core capabilities, objectives, and technical definitions to provide a clear roadmap for the development of the project.

1.1. PURPOSE

The purpose of this document is to provide a clear and structured definition of the system to support its analysis, design, and implementation phases for its development.

This document guides to developers who responsible for implementing the system, designers who responsible for system architecture and modeling, testers who responsible for validating system functionality, and stakeholders which includes the instructors who will evaluate the project by including all necessary functional and non-functional requirements necessary for system's successful design, implementation and verification.

This document ensures that all stakeholders have a shared understanding of the project's goals, ensuring clear communication, and serving as a baseline for the development lifecycle.

1.2. SCOPE

This subsection defines the scope of the software product described in this document that is the PathWise Travel Recommendation System, a web-based application designed to generate personalized travel plans based on user preferences, with mentioning the capabilities, objectives, and constraints of the system.

1.2.1. SYSTEM CAPABILITIES

The PathWise Travel Recommendation System provides personalized travel plans by user-defined parameters such as destination, budget, number of travelers, and travel duration. By matching user preferences and tags with specific venue attributes, the system generates at least three alternative travel plans that filtering constraints. Users can compare alternative plans, select their preferred plans, and provide feedback through a rating system to improve future recommendations.

The system generates recommendations based on tag compatibilies, location proximities between venues, and the user-defined parameters. The system supports role-based interaction model where business owners can manage their venues and administrators review and approve venues submitted by business owners before they are published to the public.

1.2.2. SYSTEM OBJECTIVES AND GOALS

The PathWise system is designed to provide personalized and adaptive travel recommendations, enable comparison between multiple generated plans, improve recommendation quality using user feedback mechanisms, and ensure structured role-based system interaction.

1.2.3. SYSTEM CONSTRAINTS AND LIMITATIONS

The PathWise system will run in a local development environment and use a rule-based recommendation approach. The system will not support real-time booking or payment processing, integrate with external APIs or third-party services, and operate under high scalability or distributed system conditions.

1.3. DEFINITIONS, ACRONYMS, AND ABBREVIATIONS

Table 1: *Definitions, Acronyms, and Abbreviations*

Term / Acronym	Definition
SRS	Software Requirements Specification document that defines system requirements
API	Application Programming Interface used for communication between system components
RDBMS	Relational Database Management System used for structured data storage
ORM	Object-Relational Mapping technique used to interact with the database

Term / Acronym	Definition
Tag	A user-selected or system-inferred attribute representing user preferences, used for matching users with venues
Recommendation	A generated travel plan consisting of one or more venues, organized according to user preferences and system parameters
Venue	A location such as a hotel, restaurant, or attraction with associated attributes
Rating	A user-provided evaluation used to assess and improve recommendation quality

1.4. REFERENCES

[1] CSE3044 SRS Template.

[2] IEEE Guide to Software Requirements Specification (ANSI/IEEE Std. 830-1984).

1.5. OVERVIEW

This Software Requirements Specification (SRS) document is organized into the following main sections:

Section 1 – Introduction: Provides the purpose, scope, definitions, references, and overall structure of the document.

Section 2 – General Description: Describes the overall system perspective, product functions, user characteristics, constraints, and assumptions .

Section 3 – Specific Requirements: Defines detailed system requirements including: External interface requirements, Functional requirements, Non-functional requirements, Design constraints, Logical database requirements.

Section 4 – UML Diagrams: Includes system modeling diagrams such as use case diagrams and sequence diagrams .

2. GENERAL DESCRIPTION

This section provides a high-level overview of the factors and requirements that influence the development and functionality of the PathWise system. It focuses on how the product interacts with its environment and the primary goals it aims to achieve for its stakeholders.

2.1. PRODUCT PERSPECTIVE

PathWise is an independent, standalone web-based system. It is not a sub-component of a large existing software product. It is designed to replace traditional, manual travel planning methods by introducing a dynamic, intelligent ecosystem The system operates on a client-server architecture, utilizing a centralized relational database to ensure persistent data management and execute complex recommendation algorithms.

PathWise is designed as an independent alternative to existing global aggregators like Trivago and Book.com. While these platforms focus on mass-market search results, PathWise distinguishes itself by integrating a customized recommendation engine that processes user-specific tags and historical preferences to create a structured "top-3" itinerary. The system is intended to coexist with these platforms by potentially utilizing their public APIs for real-time price comparisons while maintaining its unique focus on automated travel planning.

2.2. PRODUCT FUNCTIONS

The PathWise system provides several core functions designed to automate and personalize the travel planning experience. The software performs the following primary operations:

- Role Management: Secure authentication and role-specific interfaces for travelers, business owners, and administrators.
- Dynamic Travel Plan Generation: Processing user inputs (budget, duration destination etc.) to create optimized travel schedules.
- Smart Alternative Selection: Providing personalized venues alternatives using a tag-based weighting system derived from user history and preferences.
- Inventory Management: Allowing business owners to create, update, and manage their venue details, capacities, and descriptive tags.

- Ecosystem Moderation: Enabling administrators to approve or reject submitted venues to maintain data quality.

2.3. USER CHARACTERISTICS

The PathWise system is designed to accommodate a diverse range of stakeholders, each with distinct levels of technical proficiency and specific operational goals within the venue management ecosystem. The system is designed for three distinct user groups with varying levels of technical expertise:

- Standard Users: Everyday travelers seeking customized trip plans. They are assumed to possess basic digital literacy. The user interface for this group must be highly intuitive, requiring no prior training to navigate the planning templates.
- Business Owner: Hospitality and dining managers who provide venue data. They are expected to have moderate computer skills, sufficient to operate data-entry forms, manage pricing and assign tags to their venues.
- Administrator: System moderators overseeing the platform. They are expected to have a higher level of technical proficiency handle data validation, monitor user accounts, and maintain system integrity.

2.4. GENERAL CONSTRAINTS

- The system must be implemented and tested within the constraints of a single academic semester.
- The recommendation algorithm must produce functional and calid trip plans without excessive hardware resource consumption.
- All major business logic components must be highly modular to ensure unit testability.

2.5. ASSUMPTIONS AND DEPENDENCIES

- The system relies on a continuous internet connection for user interaction and database updates.
- It is assumed that business owners will input accurate and up-to-date capacity, price and tag information.

- The platform depends on a reliable relational database to maintain persistent records of users, venues and historical recommendation weights.

3. SPECIFIC REQUIREMENTS

This section provides a detailed and structured description of the requirements for the PathWise system. It focuses on defining the essential functionalities and operational constraints necessary for the development.

3.1. EXTERNAL INTERFACE REQUIREMENTS

This subsection defines the logical and physical connections between the PathWise system and its external environment. It specifies the requirements for user interaction, hardware dependencies, and integration with existing software or communication protocols.

3.1.1. USER INTERFACES

This subsection describes the essential user interfaces that facilitate the core operations of the PathWise system. Each interface is defined by its requirements and design elements, supported by visual mockups aimed at optimizing user interaction and accessibility across the platform.

3.1.1.1. LOGIN PAGES

The login page shall allows users to access the system using their registered credentials. The system provides separate login interfaces for administrators and standard users/business owners. Main interface components include followings:

- Username or e-mail input field
- Password input field
- Login button
- Navigation options for user or business owner login

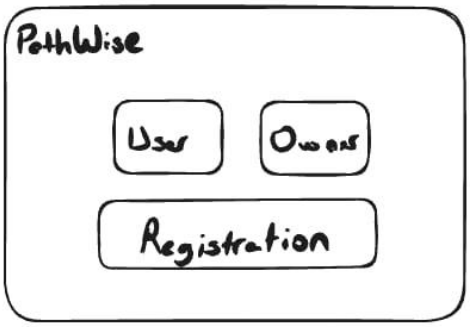


Figure 2: *Login Screen*

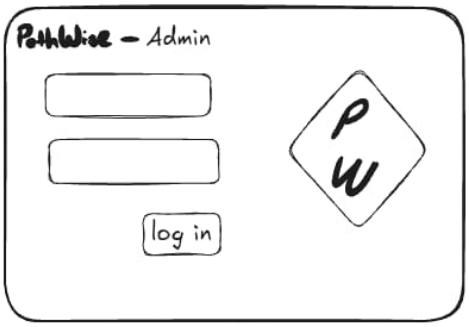


Figure 2: *Admin Login Screen*

The system verifies user credentials and redirects the user to appropriate dashboard based on their role.

3.1.1.2. REGISTRATION PAGE

The registration page shall allows new users to create an account in the system. Users must provide the required personal information to register. After successful registration, the user account is stored in the system database. Main interface components include followings:

- Name or username field
- Email field
- Password field
- User type selection (e.g., Standard User or Business Owner)
- Registration button

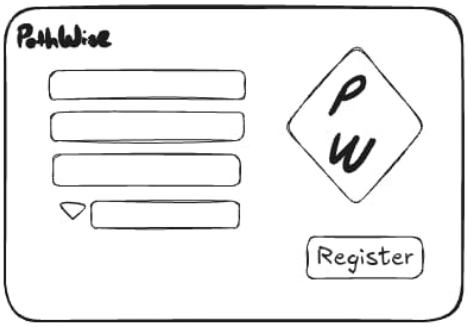


Figure 3: *Registration Screen*

3.1.1.3. ADMIN PANEL PAGE

The admin panel shall provides administrators with tools to manage the platform and monitor system activity. Main functionalities include followings:

- Viewing and managing user accounts
- Viewing and managing business owner accounts
- Reviewing and moderating venue submissions
- Accessing system notifications
- Registration button

The administrator can perform moderation actions such as approving or rejecting venue enties.

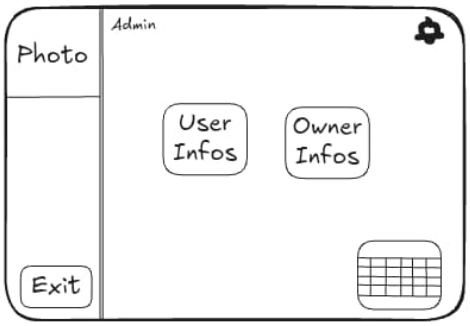


Figure 4: *Admin Panel*

3.1.1.4. USER DASHBOARD

The User Dashboard is the main interface for standard users after logging into the system. This page shall allows users to manage their travel plans and interact with the recommendation system. Main components include followings:

- Search bar for travel destinations
- Area for stared venues
- Display section for previously generated trips
- Navigation options to other system pages

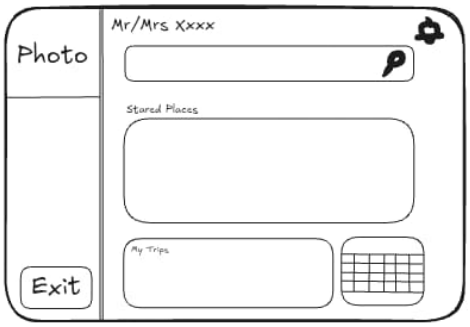


Figure 5: *User Dashboard*

3.1.1.5. OWNER DASHBOARD

The Owner Dashboard shall provides business owners with access to venue management functionalities. Main features include followings:

- Viewing previously submitted venues
- Applying to add new venues to the system

- Managing venue information and updates

Business owners can submit new venue entries which will later be reviewed by administrators.

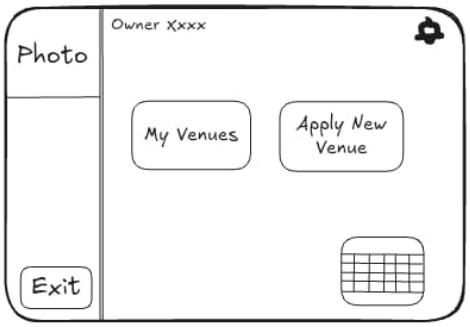


Figure 6: *Owner Dashboard*

3.1.1.6. RECOMMENDATION PAGE

The Recommendation Page shall allows users to generate personalized travel recommendations. Users can enter travel-related parameters to receive suggested venues. Main interface components include followings:

- Destination input field
- Travel date inputs
- Budget input field
- Number of travelers
- Recommendation search button

The system processes these inputs and generates personalized travel suggestions.

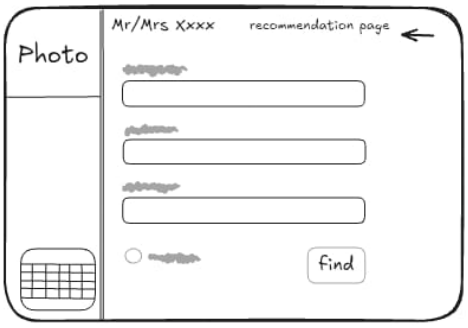


Figure 7: *Recommendation Page*

3.1.1.7. BUSINESS VIEW PAGE

The Business View Page shall allows administrators to review business-related information within the system. Main functionalities include followings:

- Viewing submitted business entries
- Monitoring notifications related to businesses
- Managing approval status of submitted venues

This interface helps administrators maintain system data quality.

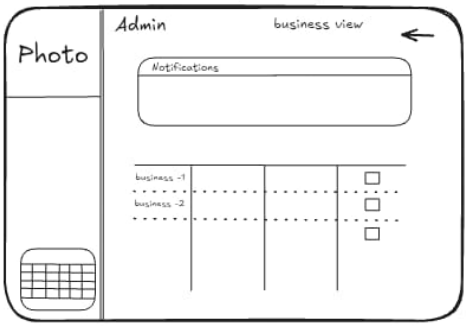


Figure 8: *Business View Page*

3.1.1.8. VENUE ADDING PAGE

The Venue Adding Page shall allows business owners to submit new venues to the system. Main components include followings:

- Venue information fields
- Image upload section
- Venue description inputs
- Submission button

After submission, the venue will be sent to the administrator for approval.

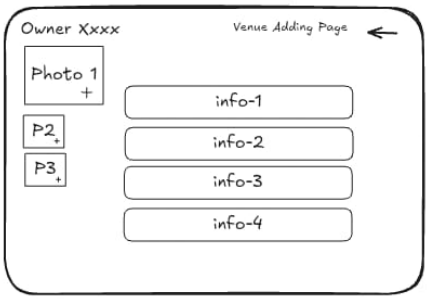


Figure 9: *Venue Adding Page*

3.1.1.9. VENUE SEARCH PAGE

The Venue Search Page shall allows users to search for venues. Main interface components include followings:

- Search input field
- List of available and relevant venues
- Venue selection options

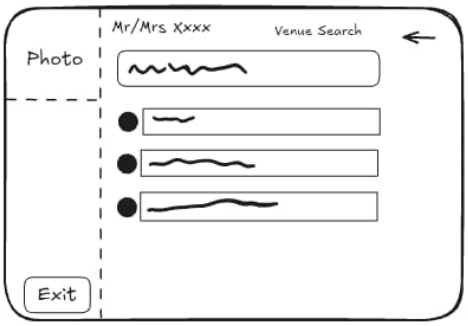


Figure 10: *Venue Search Page*

3.1.2. HARDWARE INTERFACES

The PathWise system shall not require specialized hardware components. The system is designed as a web-based application and can operate on standard personal computing devices. Users access the system through common devices such as:

- Desktop computers
- Laptop computers
- Tablets or smartphones with a web browser

Minimum recommended hardware specifications include:

- Processor: Dual-core CPU or equivalent
- Memory: Minimum 4 GB RAM
- Storage: At least 500 MB available space

The server-side components of the system may run on a standard development machine capable of running the backend framework and database system.

3.1.3. SOFTWARE INTERFACES

The PathWise system interacts with several software components that support the operation of the application.

3.1.3.1. FRONTEND INTERFACES

The user interface is implemented using React, a JavaScript library used for building interactive web interfaces. The frontend application runs in a web browser and communicates with the backend server using HTTP requests.

3.1.3.2. BACKEND INTERFACES

The backend services are implemented using ASP.NET Core Web API written in C#. This backend component handles user authentication, business logic, travel recommendation processing and venue management operations.

3.1.3.3. DATABASE INTERFACES

The system stores persistent data in a relational database management system. The backend communicates with the database through Entity Framework Core, which provides an object-relational mapping layer. The database stores user accounts, venue information travel preferences and it records related to recommendations.

3.1.4. COMMUNICATION INTERFACE

Communication between system components occurs through standard web communication protocols. The frontend and backend components communicate using the HTTP/HTTPS protocol through RESTful API endpoints. Data exchanged between the client and server is structured using the JSON format. The typical communication flow is as following:

1. The user interacts with the React-based frontend interface.
2. The frontend sends an HTTP request to the backend API.
3. The backend processes the request and performs necessary operations such as database queries or recommendation processing.
4. The backend returns a response to the frontend in JSON format.
5. The frontend displays the processed information to the user.

The system may also use secure HTTPS connections to ensure safe communication between the client and server.

3.2. FUNCTIONAL REQUIREMENTS

This subsection defines the specific services, tasks, or functions that the system must perform. It describes how the software should respond to particular inputs and how it should behave in specific situations to meet user needs.

3.2.1. FR #1: USER AUTHENTICATION AND AUTHORIZATION

3.2.1.1. DESCRIPTION

The system shall authenticate users via their registered credentials and authorize access to specific functionalities based on their assigned role as a Standard User, Business Owner, or Administrator.

3.2.1.2. INPUTS

- User Identifier: The e-mail of the user.
- Security Credential: The password of the user.
- System Trigger: A UI action, specifically clicking the "Login" button.

3.2.1.3. PROCESSING

1. Validation: Format of the credentials are validated.
2. Data Retrieval: Corresponding user data is retrieved from the database.
3. Verification: Input and the retrieved data is compared and accordingly decide to grant access if they match or generate an authentication error if they do not match.
4. Authorization: Grant access to the user based on the role and the account status of the user.
5. Session Management: Create an authenticated session and issue a token.
6. Redirection: Redirect the user to the corresponding dashboard based on the role.

3.2.1.4. OUTPUTS

- Authentication Token: A secure session token that contains user identity and role information.
- Redirection: Redirection to appropriate user dashboard on success.

- Error Message: A descriptive notification to the user in case of failure.

3.2.1.5. CONSTRAINTS

- Encryption: Passwords shall never be stored or transmitted in plain text, they must be stored as salted hashes using a strong algorithm.
- Latency: The entire authentication process shall be completed within 2 seconds under normal network connections.
- Security Lockout: The system shall temporarily lock the account after 5 consecutive failed login attempts with a rate-limiting mechanism to prevent brute-force attacks.
- Session Timeout: Authenticated sessions shall automatically expire after 15 minutes of inactivity.

3.2.1.6. ERROR HANDLING

- Invalid Credentials: If the input e-mail and password combination is not matching with the corresponding user data, system displays an error message.
- Missing or Incomplete Input Fields: If the user does not fill in the e-mail or password field, system displays an error message.
- Locked Account: If the account is locked due to security lockout, system displays an error message.
- Unauthorized Access: If the system detects an unauthorized access attempt, system displays an error message.
- Expired Session: If the user attempts an action with an expired token, system displays an error message and redirects the user to login page.
- Database Connection Failure: If the system can not access the database, system displays an error message.

3.2.2. FR #2: TRAVEL PLAN GENERATION

3.2.2.1. DESCRIPTION

The system shall generate a chronological set of travel recommendations by analyzing venue data and user preference tags against specific parameters, including destination, travel dates, budget, and the number of travelers, to assist the user in travel planning.

3.2.2.2. INPUTS

- Destination: The specific city or region.
- Travel start and end dates: The calendar range for departure and return.
- Budget: Planned spending limit.
- Number of travelers: Number of people joining the travel.
- User preference tags: Keywords to tailor the recommendation.
- System Trigger: A UI action, specifically clicking the "Create Travel Plan" button.

3.2.2.3. PROCESSING

1. Validation: Format of the inputs are validated.
2. Data Retrieval: Search for available hotels and venues located in the specified destination that is not exceeding the input budget.
3. Preference Matching: Use the input tags and user's stored preference preferences those are the historic data and accordingly eliminate non-matching hotels and venues.
4. Ranking: Sort the remaining candidates based on the similarity score between the venue features and user preferences.
5. Scheduling: Organize recommended venues chronologically within the travel period.
6. Cost Calculation: Estimate the total cost of the proposed travel plan.

3.2.2.4. OUTPUTS

- Personalized Plans: A list of recommended travel plans those are personalized which includes suggested accomodation options and chronological order of recommended activities.
- Total Costs: Estimated travel costs for each plan.
- Error Message: A descriptive notification to the user in case of failure.

3.2.2.5. CONSTRAINTS

- Budget Limit: The generated recommendations must not exceed the input budget.
- Admin Approval: Only venues approved by administrators may be included in recommendations.
- Capacity Match: Venue availability and space capacity must satisfy the total number of travelers provided in the input.

- Date Consistency: All recommended activities and accommodations must fall strictly within the user's selected travel dates.

3.2.2.6. ERROR HANDLING

- Missing or Incomplete Input Fields: If the user does not fill in required fields such as destination or travel dates, system displays an error message.
- No Available Venues: If there are no venues matching the selected destination in the database, system displays an informative error message.
- Database Connection Failure: If the system cannot access the venue or user preference data from the database, system displays an error message.

3.2.3. FR #3: VENUE MANAGEMENT BY BUSINESS OWNERS

3.2.3.1. DESCRIPTION

The system shall allow Business Owners to manage venue information within the system. Business Owners can create, update, and delete venue entries associated with their accounts.

3.2.3.2. INPUTS

- Venue Data: Venue name, category, price range, capacity, location, and descriptive tags.
- Media: Optional images or descriptions.
- System Trigger: A UI action for creating, editing, or deleting a venue.

3.2.3.3. PROCESSING

1. Validation: The system shall validate all input fields and required attributes.
2. Storage: The system shall store the venue information in the database.
3. Status Assignment: The system shall mark newly created venues as "Pending" for administrator approval.
4. Modification: The system shall allow owners to update or delete existing venue records.

3.2.3.4. OUTPUTS

- Confirmation Message: A notification after successful creation, update, or deletion.

- Venue List: An updated display of venues owned by the Business Owner.
- Status Indicator: Visual feedback showing if a venue is “Approved”, “Pending”, or “Rejected”.

3.2.3.5. CONSTRAINTS

- Authentication: Only authenticated Business Owners shall manage venue entries.
- Required Fields: Each venue shall include mandatory attributes (category, price range, and capacity).
- Approval Gate: Newly added venues shall remain invisible to users until approved by an Administrator.

3.2.3.6. ERROR HANDLING

- Missing Information: If required fields are empty, the system shall display an error message.
- Unauthorized Access: If a non-owner attempts to edit a venue, the system shall block the action and display an error.
- Invalid Format: If data formats (e.g., price) are incorrect, the system shall display a validation error.

3.2.4. FR #4: ALTERNATIVE VENUE RECOMMENDATION

3.2.4.1. DESCRIPTION

The system shall provide alternative venue recommendations when a user requests to replace a specific venue within an existing travel plan.

3.2.4.2. INPUTS

- Target Venue: The specific venue selected for replacement.
- Plan Context: The existing travel plan data (dates, location, budget).
- System Trigger: A UI action to “Replace Venue”.

3.2.4.3. PROCESSING

1. Identification: The system shall identify the category and attributes of the venue to be replaced.
2. Retrieval: The system shall fetch candidate venues of the same category from the database.
3. Filtering: The system shall filter candidates based on remaining budget and availability.
4. Matching: The system shall compare venue tags with the user's preference profile.
5. Scoring & Ranking: The system shall calculate similarity scores and rank the alternatives.

3.2.4.4. OUTPUTS

- Ranked List: A set of alternative venues sorted by relevance.
- Venue Details: Comprehensive information (price, tags, category) for each suggestion.

3.2.4.5. CONSTRAINTS

- Category Consistency: Alternative venues shall belong to the same category as the original.
- Budget Alignment: Recommendations shall respect the user's total travel budget.
- Approval Status: Only administrator-approved venues shall be recommended.

3.2.4.6. ERROR HANDLING

- No Results: If no suitable alternatives are found, the system shall display a "No alternatives available" message.
- Data Corruption: If travel plan data is missing, the system shall display an error message

3.2.5. FR #5: ADMINISTRATIVE CONTENT MODERATION

3.2.5.1. DESCRIPTION

The system shall enable administrators to review and moderate venue entries submitted by Business Owners to ensure they meet system standards.

3.2.5.2. INPUTS

- Decision: Approval or rejection status.

- Moderation Notes: Optional feedback from the administrator.
- System Trigger: Clicking "Approve" or "Reject" buttons in the admin panel.

3.2.5.3. PROCESSING

1. Retrieval: The system shall fetch all venue entries marked as "Pending Approval."
2. Review Display: The system shall present full venue details to the Administrator.
3. Status Update: The system shall update the venue status based on the decision.
4. Database Commit: The system shall store the decision and any notes in the database.

3.2.5.4. OUTPUTS

- Action Confirmation: A message confirming the status change.
- Status Update: The venue's status shall change to "Approved" or "Rejected" instantly.
- Visibility Update: Approved venues shall be added to the recommendation pool.

3.2.5.5. CONSTRAINTS

- Admin Only: Only users with the "Administrator" role shall access moderation functions.
- Visibility: Rejected venues shall never be visible to Standard Users.

3.2.5.6. ERROR HANDLING

- Unauthorized Attempt: The system shall block moderation access for non-admin users.
- Invalid Action: If an administrator attempts to approve a venue with missing data, the system shall display an error.

3.2.6. FR #6: USER ACCOUNT DELETION

3.2.6.1. DESCRIPTION

The system shall allow Standard Users to permanently delete their accounts and all associated personal data.

3.2.6.2. INPUTS

- Deletion Request: User's intent to delete the account.

- Confirmation: A secondary confirmation of the action.

3.2.6.3. PROCESSING

1. Verification: The system shall verify the user's active session.
2. Confirmation Prompt: The system shall display a warning regarding the permanent nature of the action.
3. Data Erasure: The system shall remove all account data, preference profiles, and history from the database.
4. Session Termination: The system shall immediately terminate the user's session.

3.2.6.4. OUTPUTS

- Final Notification: A confirmation message before the user is logged out.
- Logout: Automatic redirection to the landing or login page.

3.2.6.5. CONSTRAINTS

- Irreversibility: Account deletion shall be final and non-recoverable.
- Self-Service: Users shall only be able to delete their own accounts.

3.2.6.6. ERROR HANDLING

- Database Failure: If the deletion fails due to a system error, the system shall display a “Try again later” message.
- Invalid Request: The system shall block unauthorized deletion attempts.

3.2.7. FR #7: PREFERENCE PROFILE RESET

3.2.7.1. DESCRIPTION

The system shall allow users to reset their stored preference profiles, removing all historical tag weights to start as a new user.

3.2.7.2. INPUTS

- Reset Request: A UI trigger to clear preferences.
- Confirmation: Confirmation of the reset action.

3.2.7.3. PROCESSING

1. Identification: The system shall identify the authenticated user's profile.
2. Data Removal: The system shall delete all stored tag weights and preference history.
3. State Initialization: The system shall update the user profile to an "Empty" preference state.

3.2.7.4. OUTPUTS

- Confirmation Message: A notification that the profile has been reset.
- Clean Profile: The system shall display a blank state for personalized recommendations.

3.2.7.5. CONSTRAINTS

- Authenticated Only: Only logged-in users shall reset their own profiles.
- Algorithm Behavior: Post-reset, the recommendation system shall treat the user as a new entity.

3.2.7.6. ERROR HANDLING

- Partial Deletion: If tag weights fail to clear, the system shall display an error message and retain previous data.
- Unauthorized Attempt: The system shall block attempts to reset another user's profile.

3.3. NON-FUNCTIONAL REQUIREMENTS

This subsection defines the quality attributes, design constraints, and external interface requirements that the system must adhere to. These requirements specify the criteria used to judge the operation of the system as a whole.

3.3.1. PERFORMANCE

The PathWise system shall provide efficient and responsive system performance during user interactions.

- The system shall respond to standard user actions such as login, venue search, and profile access within 2 seconds under normal operating conditions.
- The dynamic travel plan generation function shall produce a personalized travel plan within 3 seconds after receiving user parameters such as destination, travel dates, and budget.
- Requests for alternative venue recommendations shall return results within 3 seconds.
- The system shall support multiple simultaneous user operations such as travel plan generation and venue submission without noticeable performance degradation.

3.3.2. RELIABILITY

The system shall operate consistently and preserve system data integrity.

- User accounts, venue entries, and travel preference profiles shall be stored persistently in the system database.
- System failures shall not cause loss of critical user data such as stored venue information or user profiles.
- Operations such as venue approval, venue deletion, and user account deletion shall be executed as atomic transactions.
- The system database shall be backed up at least once every 24 hours.

3.3.3. AVAILABILITY

The PathWise system shall remain accessible to users during system operation.

- The system shall maintain at least 95% system availability during operational periods.

- The system shall continue functioning when multiple users simultaneously request travel plans or submit venue data.
- After a system restart or maintenance event, the system shall become operational again within two minutes.

3.3.4. SECURITY

The PathWise system shall ensure secure access control and protection of user information.

- The system shall authenticate users using registered credentials and assign them roles such as Standard User, Business Owner, or Administrator.
- Access to system functions shall be restricted based on the permissions associated with each user role.
- User passwords shall be stored using secure cryptographic hashing mechanisms and shall not be stored in plain text.
- Administrative operations such as venue approval or rejection shall only be accessible to authorized administrators.
- The system shall validate all user inputs to prevent invalid or malicious data from entering the system.

3.3.5. MAINTAINABILITY

The PathWise system shall be designed to support future updates and system improvements.

- The system architecture shall follow a modular design, separating components such as user management, venue management, recommendation logic, and administrative functions.
- System components shall be loosely coupled to allow modifications without affecting unrelated modules.
- Source code shall follow consistent documentation and naming conventions to facilitate maintenance.

3.3.6. PORTABILITY

The PathWise system shall operate across common computing environments.

- The system shall run on commonly used operating systems including Windows, Linux, and macOS.
- The system shall be accessible through modern web browsers such as Google Chrome, Mozilla Firefox, and Microsoft Edge.
- The web interface shall adapt to common desktop screen resolutions.

3.4. INVERSE REQUIREMENTS

This subsection specifies the constraints and boundaries of the system by defining what the system shall not do. These inverse requirements are essential to prevent scope creep, ensure data security, and maintain the integrity of the business logic by explicitly excluding unauthorized actions or unintended system behaviors.

- The system shall not store user passwords in plain text; all credentials must be salted and hashed using industry-standard algorithms.
- The system shall not allow a Standard User to access or modify any data belonging to the Administrator or Business Owner panels.
- The system shall not suggest venues or hotels that are located outside the user's specifically selected destination area.
- The system shall not share or sell user-provided preference tags to any third-party services without explicit user authorization.
- The system shall not process a reservation if the user's selected dates overlap with an existing booking in the same account.

3.5. DESIGN CONSTRAINTS

This subsection outlines the specific technical limitations, technology stack requirements, and architectural standards that must be strictly followed during the development of the PathWise system. These constraints ensure that the software remains maintainable, secure, and compatible with the intended local development and web-based runtime environments.

3.5.1. TECHNOLOGY STACK CONSTRAINTS

The system shall be implemented using the following technologies:

- React for the frontend user interface
- ASP.NET Core Web API for backend services
- C# programming language
- A relational database management system (RDBMS)

These technologies are selected to ensure compatibility between system components.

3.5.2. WEB-BASED ARCHITECTURE

The system must operate as a web-based application accessible through standard web browsers. No dedicated desktop software installation should be required for users.

3.5.3. LOCAL DEVELOPMENT ENVIRONMENT

The system will initially operate in a local development environment using a local server and local database. This constraint allows the development team to test and demonstrate the system without requiring cloud deployment.

3.5.4. DATA INTEGRITY

All system data must be stored in a structured relational database to maintain consistency between users, venues, and recommendation data.

3.5.5. SECURITY CONSTRAINTS

Basic authentication mechanisms must be implemented to ensure that:

- only registered users can access personalized recommendations
- only business owners can manage their venue data
- only administrators can approve newly added venues

3.6. LOGICAL DATABASE REQUIREMENTS

The system requires a structured relational database to store and manage all application data. The database will be designed following relational database principles to ensure data consistency, integrity, and efficient querying.

The logical database structure is derived from the system's domain model and functional requirements. The main data entities include users, venues, recommendations, and related metadata such as tags. The database will maintain relationships between users, venues, and recommendation results generated by the system

3.6.1. User Data

The system must store information about registered users. Each user record contains basic account information and role identification.

Attributes include:

- userID (unique identifier)
- name
- mail
- password (encrypted)
- role (StandardUser, BusinessOwner, Admin)

The role attribute determines the permissions and system access level for each user.

3.6.2. Venue Data

The system must store information about venues submitted by business owners.

Attributes include:

- venueID (unique identifier)
- venueName
- location
- category
- tags
- ownerID (reference to the business owner)

Venues are associated with business owners and may contain descriptive tags that support the recommendation system

3.6.3. Recommendation Data

The system stores recommendation results generated for users.

Attributes include:

- recommendationID

- userID
- generatedDate

Each recommendation may contain multiple venues suggested by the system

3.6.4. Tag Data

Tags are used to categorize venues and support filtering and recommendation generation.

Attributes include:

- tagID
- tagName

Venues may have multiple associated tags

3.6.5. Data Relationships

The logical database must maintain the following relationships:

- A **BusinessOwner** can manage multiple **Venues**
- A **User** can receive multiple **Recommendations**
- A **Recommendation** can include multiple **Venues**
- A **Venue** and **User** can contain multiple **Tags**

These relationships allow the system to efficiently generate personalized recommendations and manage venue information.

4. UML DIAGRAMS

This section presents the core artifacts produced during the system analysis phase. These models provide a comprehensive view of the system's functional requirements, behavioral dynamics and structural organization through various diagrams and detailed specifications.

4.1. USE CASES

This subsection outlines the functional requirements of the system through fully dressed use cases and their corresponding UML diagrams. These artifacts define the interactions between the actors and the system to achieve specific goals.

4.1.1. USE CASE #1

Use Case UC1: User Registration

Scope: PathWise System.

Level: User Goal.

Primary Actors: User.

Stakeholders and Interests:

- Prospective Guest: Wants to access the PathWise system in order to be able to search for destinations and generate personalized travel plans.
- Business Owner: Wants to access the PathWise system in order to be able to list their hotel details, making them discoverable by prospective guests and to manage bookings efficiently.
- Administrator: Wants Business Owner registration requests to be routed for review so that only verified businesses gain access to listing capabilities.
- System: Must ensure that only valid and unique accounts created and secured.

Preconditions:

- PathWise system is operational and accessible.
- User has accessed the registration page of the website.

Success Guarantees:

- User is notified about the outcome (success, failure or pending approval) of the registration process.
- On Business Owner registration, the request is queued for Administrator review.

Main Success Scenario:

1. System requests the required registration information.
2. User provides the required registration information: name, surname, e-mail, password and registration type (Standard User or Business Owner).
3. System validates the entered information.
4. System creates the user account.

Extensions:

2a. User provides incomplete registration information.

   1. System displays a warning message and highlights the missing fields with using inline validation.

2b. User provides an e-mail address that is already associated with an existing account.

   1. System displays an error message to the user and highlights the e-mail field.

2c. User provides the required information in an invalid format (e.g. missing '@' character in e-mail field, a password that is too short).

   1. System displays an error message to the user and highlights the invalid fields.

4a. In the case of "Business Owner" was selected as the registration type, system creates the user account as a pending approval account.

   1. The registration request of the pending approval account is forwaded to the Administrator's dashboard for review.
   2. If the Administrator approves the request, the account is fully activated.
   3. If the Administrator rejects the request, the account is not created and user is notified.

Special Requirements:

- Performance: The inline validation feedback provided by the UI must appear within 750 milliseconds of the user finishing input in a field, ensuring a responsive and non-disruptive experience.
- Reliability: If the registration fails, system must not create a corrupted account record.
- Availability: The registration page must be available 99.9% of the time.
- Security: Passwords should have a strength policy and they must never be stored in a plain text, they must be stored as salted hashes using a strong algorithm.
- Maintainability: Validation rules must be configurable without requiring major code changes.
- Portability: The registration form must be visually consistent across modern browsers such as Chrome and Edge.

Technology and Data Variations List:

- Registration type: Standard User, Business Owner.
- E-mail validation: RFC 5322-compliant format check.
- Password storage: BCrypt, MD5, SHA-256.

Frequency of Occurrence: Could be nearly continuous.

Miscellaneous:

- Should business owner registration require supplementary documentation to be uploaded during registration to make it easy to review for administrators?
- How business owners will be informed when their registrations are approved by the administrator?

Use Case Diagram UCD1: User Registration

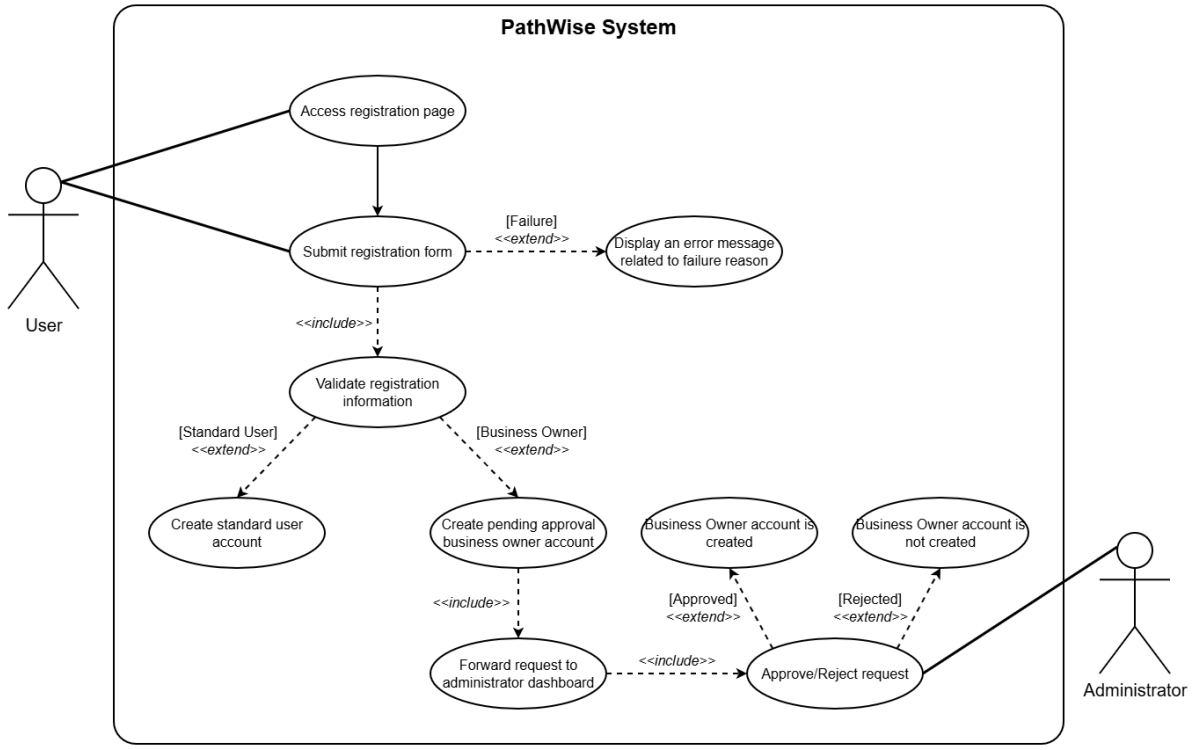


Figure 11: *Use Case Diagram #1*

4.1.2. USE CASE #2

Use Case UC2: User Login

Scope: PathWise System.

Level: User Goal.

Primary Actors: User.

Stakeholders and Interests:

- User: Wants to login their PathWise accounts to access the provided features by the PathWise system (includes all of the standard users, business owners and administrators).
- System: Must allow registered users to authenticate and access the system securely.

Preconditions:

- PathWise system is operational and accessible.
- User has accessed the login page of the website.

Success Guarantees:

- Users redirected to their dashboards.

Main Success Scenario:

1. System requests the required login information: e-mail and password.
2. User provides the required login information.
3. System forwards credentials to the authentication service.
4. Authentication service verifies the credentials.
5. Session manager creates a new authenticated session and issues a token.

Extensions:

2a. User provides incomplete login information.

   1. System displays a warning message and highlights the missing fields with using inline validation.

2b. User provides an e-mail address that is not associated with an existing account or a password that is not matching with the e-mail.

   1. System forwards credentials to the authentication service.
   2. Authentication service can not verify the credentials.
   3. User is informed about the e-mail and password are not matching.
   4. The failed login attempts counter is incremented. If it exceeds a threshold, like 5, the user will not be able to attempt another login request temporarily for a minute.

4a. Authentication service verifies the credentials, but the user is a Business Owner and the user's account is not approved by the Administrator yet.

   1. The business owner is informed about the account creation request is still pending.

Special Requirements:

- Performance: The inline validation feedback must appear within 750 milliseconds of the user finishing input in a field, ensuring a responsive and non-disruptive experience.
- Availability: The login page must be available 99.9% of the time.
- Security: There must be a rate-limiting mechanism to prevent brute force attacks.
- Portability: The registration form must be visually consistent across modern browsers such as Chrome and Edge.

Technology and Data Variations List:

- Session Management: JWT, Traditional Session Cookies.

Frequency of Occurrence: Could be nearly continuous.

Miscellaneous:

- Is it necessary to have multi-factor authentication?

Use Case Diagram UCD2: User Login

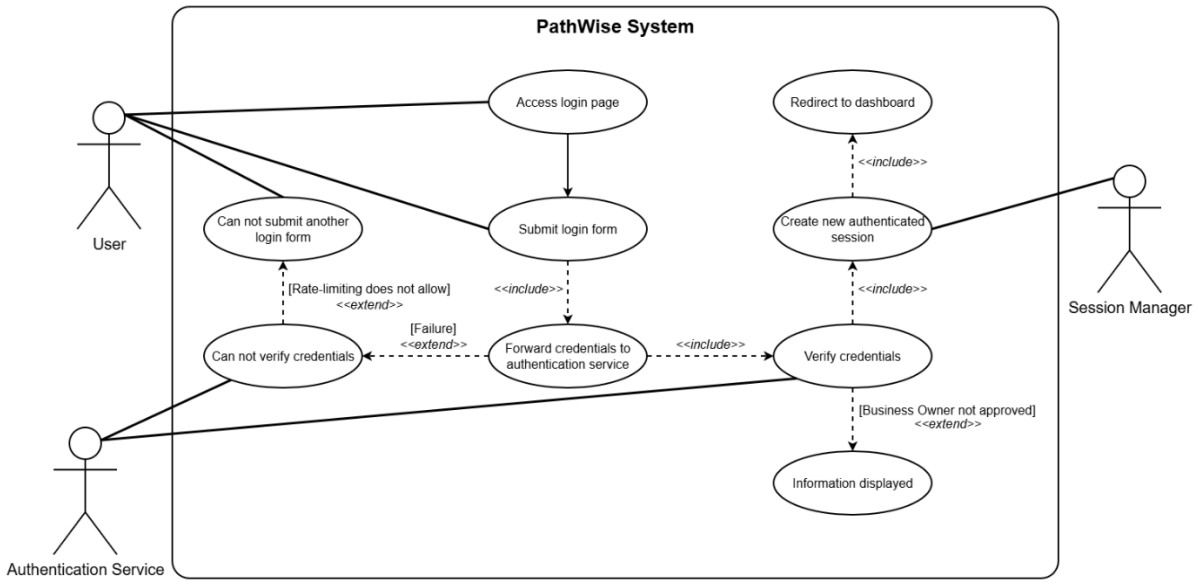


Figure 12: *Use Case Diagram #2*

4.1.3. USE CASE #3

Use Case UC3: Travel Plan Recommendation

Scope: PathWise System.

Level: User Goal.

Primary Actors: User.

Stakeholders and Interests:

- Prospective Guest: Wants to receive personalized hotel and venue recommendations based on their preferences, budget, travel dates, group size, and optionally based on their histories, without needing to manually search and compare options.
- Business Owner: Wants to their listings to appear in results when they match user criterias.
- System: Must collect user preferences accurately and list ranked, constraint-satisfying travel plans to prospective guests.

Preconditions:

- PathWise system is operational and accessible.
- User is authenticated and has accessed the recommendation page of the website.
- Hotel and venue data is available and up to date in the system.

Success Guarantees:

- User receives at least one ranked travel plan containing at least a hotel and a venue suggestion that satisfy their constraints and preferences.

Main Success Scenario:

1. User enters the required travel inputs (destination, dates, budget, travelers, and tags).
2. System validate the format of the inputs.
3. System searches for hotels and venues in the destination that do not exceed the input budget.
4. System eliminates hotels and venues based on input tags and historic data those are not matching with user choices.
5. System ranks the candidates based on similarity scores.
6. System organizes the recommended venues chronologically within the travel period.
7. System estimates the total cost of the proposed plans.
8. System displays the finalized top-3 travel plans to the user.

Extensions:

2a. System fails to valide the inputs due to user did not provide mandatory inputs for the system.

    1. System displays a warning message.

3a. System filters the results, but there is no venues matching the constraints.

1. System displays an informative message.

4a. User does not have a historic data.

1. System proceeds to eliminate venues based on provided input tags.

8a. There are less than 3 finalized plans.

1. System displays less than 3 finalized plans.

Special Requirements:

- Performance: The system shall generate and present suggestions to the user within a 10 second response time.
- Availability: The recommendation page must be available 99.9% of the time.

Technology and Data Variations List:

- Recommendation Algorithms: Collaborative Filtering, Content-Based Filtering, Hybrid Recommendation Logic.

Frequency of Occurrence: Could be nearly continuous.

Miscellaneous:

- How should the system prioritize conflicting tags?

Use Case Diagram UCD3: Travel Plan Recommendation

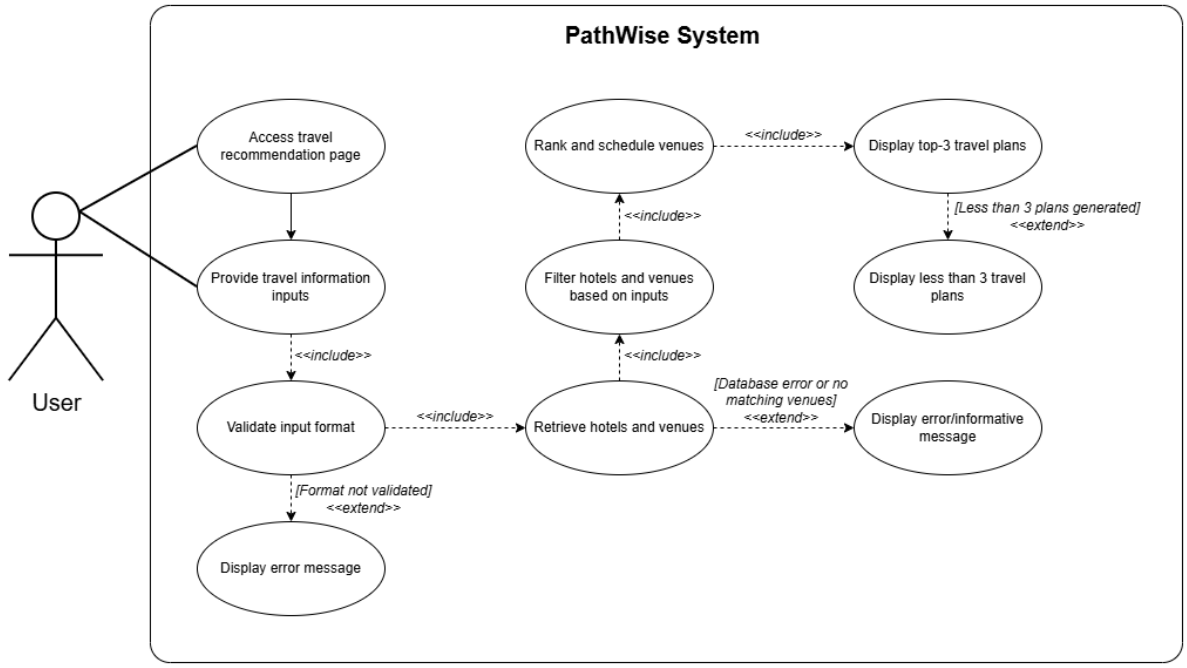


Figure 13: *Use Case Diagram #3*

4.1.4. USE CASE #4

Use Case UC4: Venue Management

Scope: PathWise System.

Level: User Goal.

Primary Actors: Business Owner, Admin.

Stakeholders and Interests:

- Business Owner: Wants to list their venues and manage them.
- Admin: Wants to confirm new venues and manage all of the venues.
- System: Must provide a secure interface for data entry, maintain the approval workflow, and ensure venue information is accurately stored in the database.

Preconditions:

- PathWise system is operational and accessible.
- The manager is authenticated as a business owner or admin.
- Existing venue records are accessible for modification or deletion.
- Hotel and venue data is available and up to date in the system.
- Venue management page is accessed.

Success Guarantees:

- Venue information is accurately created, updated, or removed in the system database.
- New or modified venues are correctly marked as pending approval until confirmed by an admin.
- An informative message about sucess or failure is displayed.

Main Success Scenario:

1. System displays a list of existing venues associated with the user.
2. User chooses the create, update or delete action for a venue.
3. System displays the appropriate input interface.
4. User enters new or updated information, and submits it to the system.
5. System validates the format of inputs.
6. System saves the changes to the database.

Extensions:

1a. There is no venues existing associated with the user.

   1. An informative message is displayed.

5a. Input validation fails.

1. User is informed.

6a. Database update fails.

1. User is informed.

Special Requirements:

- Security: Only authenticated users with the role of Business Owner or Admin shall have access to their own venue management interfaces.
- Reliability: In the event of a database failure, the system shall ensure no data corruption occurs through the use of database transactions.

Technology and Data Variations List:

- Minimum Number of Tags: 3 tags selection.

Frequency of Occurrence: Could be nearly continuous.

Miscellaneous:

- Should the admin approval process for venues be automated?

Use Case Diagram UCD4: Venue Management

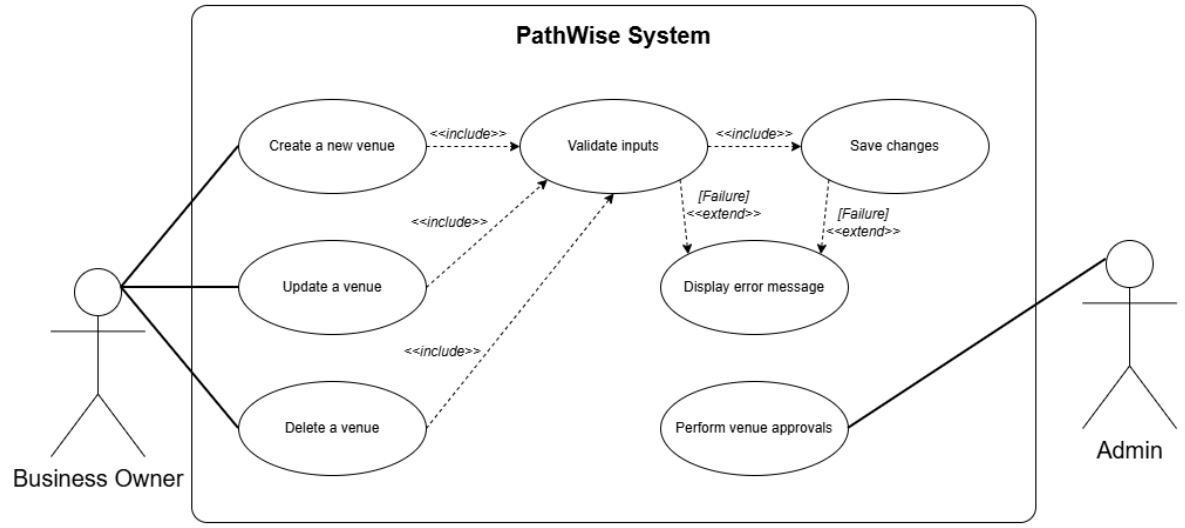


Figure 14: *Use Case Diagram #4*

4.1.5. USE CASE #5

Use Case UC5: Venue Moderation

Scope: PathWise System.

Level: User Goal.

Primary Actors: Admin.

Stakeholders and Interests:

- Administrator: Wants to review newly submitted venue entries so that only appropriate, valid and useful venues become visible in the system.
- Business Owner: Wants submitted venue entries to be reviewed fairly and promptly so they can become available to users.
- Standard User: Wants only approved and reliable venue information to appear in travel recommendations.
- System: Must preserve platform quality by preventing unreviewed or invalid venue entries from being published.

Preconditions:

- PathWise system is operational and accessible.
- Administrator is authenticated and authorized.
- At least one venue submission is in pending review status.

Success Guarantees:

- The reviewed venue is assigned a final moderation status.
- If approved, the venue becomes available for recommendation and display.
- If rejected, the venue remains unavailable to users.
- The moderation decision is stored in the system.

Main Success Scenario:

1. Administrator opens the venue moderation panel.
2. System displays the list of venue submissions awaiting approval.
3. Administrator selects a pending venue entry.
4. System displays the detailed venue information.
5. Administrator reviews the venue details.
6. Administrator chooses to approve the venue.
7. System updates the venue status as approved.
8. System makes the venue available for user searches and recommendation processes.
9. System records the moderation action.

Extensions:

2a. There are no pending venue submissions.

1. System displays an informative message indicating that there are no venues awaiting review.

4a. Venue details are incomplete or inconsistent.

1. System highlights missing or suspicious information.
2. Administrator may reject the venue submission.

6a. Administrator chooses to reject the venue.

1. System updates the venue status as rejected.
2. System prevents the venue from appearing in recommendations or listings.
3. System records the rejection decision.

6b. Administrator cancels the moderation process.

1. System leaves the venue status unchanged as pending review.

Special Requirements:

- Performance: The moderation list shall load within 2 seconds under normal local system conditions.
- Reliability: Moderation decisions must be stored atomically so that a venue cannot appear as both pending and approved/rejected.
- Availability: The moderation panel shall be available whenever the administrative subsystem is active.
- Security: Only users with the Administrator role may access moderation functions.
- Maintainability: Moderation status values and validation checks should be extendable without major redesign.
- Portability: The moderation interface should function consistently on modern desktop browsers.

Technology and Data Variations List:

- Venue status: Pending, Approved, Rejected.
- Venue categories: Hotel, Restaurant, Attraction.
- Moderation action: Approve, Reject, Cancel.

Frequency of Occurrence: Could be nearly continuous.

Miscellaneous:

- The system may optionally support a rejection reason field for better transparency toward Business Owners.
- A future version may notify Business Owners automatically after approval or rejection.

Use Case Diagram UCD5: Venue Moderation

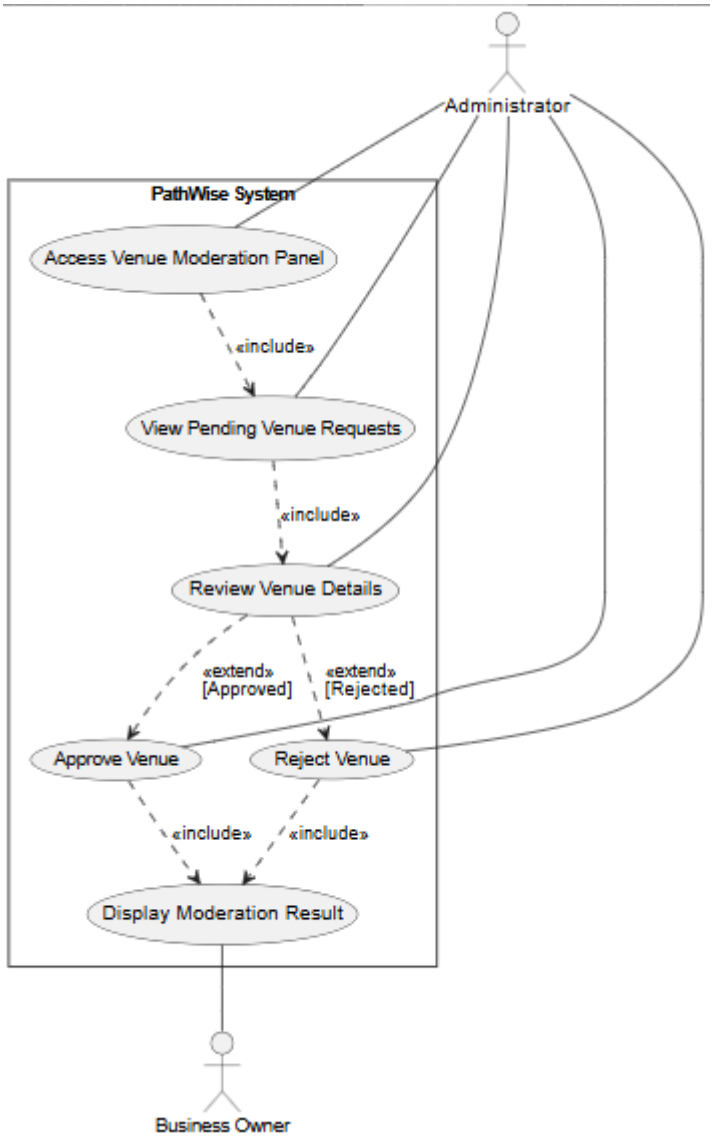


Figure 15: *Use Case Diagram #5*

4.1.6. USE CASE #6

Use Case UC6: User Account Deletion

Scope: PathWise System.

Level: User Goal.

Primary Actors: Standard User.

Stakeholders and Interests:

- Standard User: Wants to permanently remove their account and associated personal profile information from the system.
- System: Must ensure account deletion is deliberate, secure and irreversible where required.
- Administrator: Wants deletion requests to be executed safely without corrupting system data.
- Business Rules / Platform Owner: Wants user privacy to be respected while maintaining system integrity.

Preconditions:

- PathWise system is operational and accessible.
- User is authenticated.
- User has an existing account.

Success Guarantees:

- The user account is permanently deleted.
- Stored profile data and historical preference data are removed from the database.
- The deleted user can no longer authenticate with the old account.
- The system confirms the deletion result to the user.

Main Success Scenario:

1. User opens account settings.
2. System displays account management options.
3. User selects the delete account option.
4. System asks for confirmation before continuing.
5. User confirms the deletion request.
6. System deletes the user account and associated stored profile/preference data.
7. System terminates the active session.
8. System informs the user that the account has been deleted successfully.

Extensions:

3a. User cancels the deletion process.

    1. System aborts the deletion request.
    2. User account remains active and unchanged.

5a. User does not confirm the deletion request.

1. System cancels the operation and returns to the settings page.

6a. A system error occurs during deletion.

1. System rolls back the deletion transaction.
2. System keeps the account in a consistent state.
3. System informs the user that the deletion could not be completed.

6b. User session has expired before confirmation.

1. System redirects the user to the login page.
2. User must authenticate again to continue sensitive account actions.

Special Requirements:

- Performance: The deletion request shall be processed within 3 seconds under normal conditions.
- Reliability: Account deletion must be transactional; partial deletion is not acceptable.
- Availability: Account settings and deletion feature shall be available whenever the user management subsystem is online.
- Security: The system shall require an explicit confirmation step before deletion and ensure only the account owner can perform it.
- Maintainability: Deletion logic should be modular so future retention or anonymization policies can be added.
- Portability: The deletion workflow should remain usable on modern browsers and responsive interfaces.

Technology and Data Variations List:

- Confirmation Type: Confirmation button, password re-entry, or modal confirmation.
- Deletion Scope: Account only, account plus preferences, account plus recommendation history.
- Session Handling: Immediate logout after deletion.

Frequency of Occurrence: Rare.

Miscellaneous:

- In a future version, the system may provide a temporary deactivation option in addition to permanent deletion.
- Legal or institutional retention policies may affect the exact deletion scope in later revisions.

Use Case Diagram UCD6: User Account Deletion

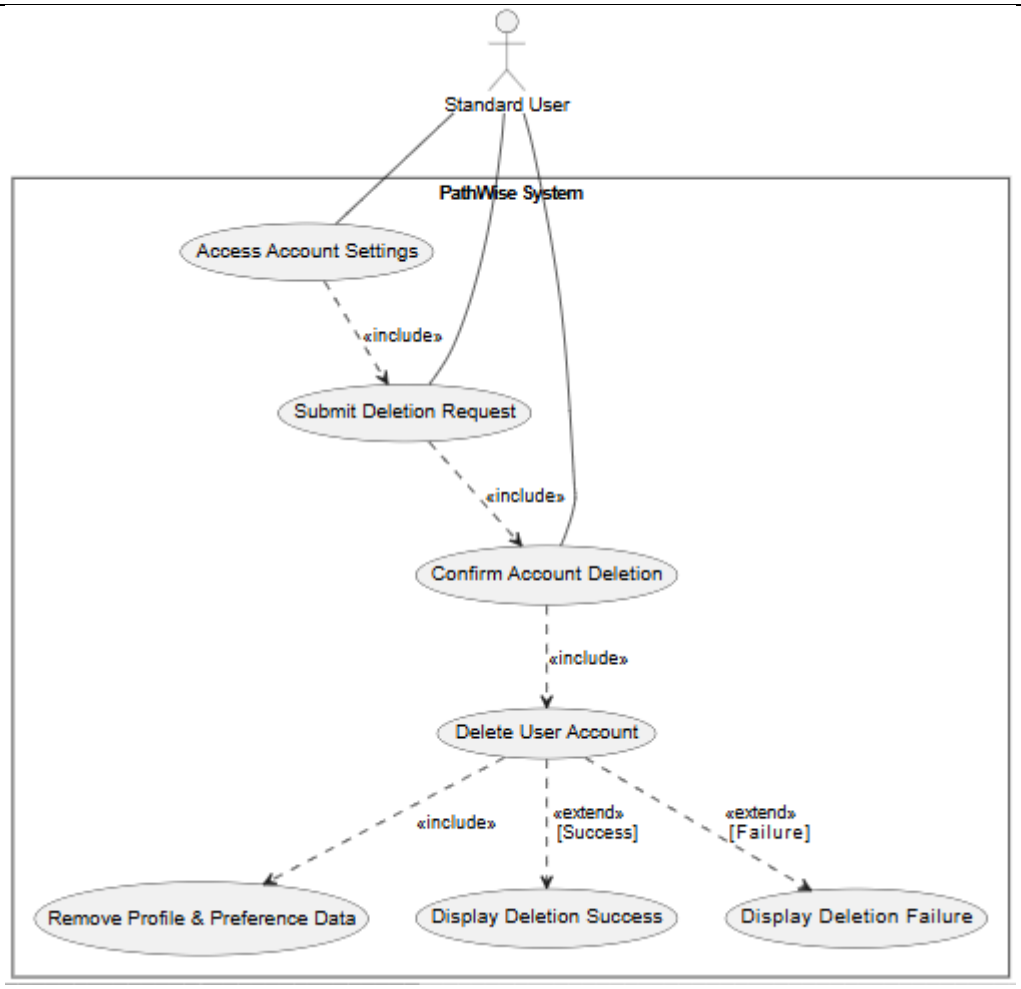


Figure 16: *Use Case Diagram #6*

4.1.7. USE CASE #7

Use Case UC7: Preference Profile Reset

Scope: PathWise System.

Level: User Goal.

Primary Actors: Standard User.

Stakeholders and Interests:

- Standard User: Wants to reset accumulated preference data so that the recommendation engine treats them as a new user.

- System: Must remove stored preference weights/history without damaging unrelated account data.
- Recommendation Engine: Needs a clean preference state to generate neutral recommendations after reset.
- Platform Owner: Wants users to remain in control of personalization behavior.

Preconditions:

- PathWise system is operational and accessible.
- User is authenticated.
- User has an existing preference profile or stored recommendation history.

Success Guarantees:

- Stored preference weights and historical preference data are removed or reinitialized.
- Future recommendations are generated without relying on previous user preference history.
- The user is informed that the reset has been completed.

Main Success Scenario:

1. User opens profile settings.
2. System displays preference management options.
3. User selects the reset preferences option.
4. System asks for confirmation.
5. User confirms the reset request.
6. System removes or reinitializes the stored preference profile data.
7. System marks the recommendation profile as reset.
8. System informs the user that future recommendations will be generated as for a new user.

Extensions:

3a. User cancels the reset operation.

    1. System aborts the process.
    2. Existing preference profile remains unchanged.

5a. User does not confirm the reset request.

    1. System returns to profile settings without modification.

6a. No stored preference profile exists.

    1. System informs the user that there is no preference data to reset.
    2. System leaves the account unchanged.

6b. A system error occurs during reset.

1. System rolls back any partial changes.
2. System preserves the existing preference profile if reset cannot be completed.
3. System informs the user of the failure.

Special Requirements:

- Performance: Preference reset shall complete within 2 seconds under normal conditions.
- Reliability: Reset must not affect authentication data, venue history unrelated to preferences, or account identity data.
- Availability: Preference reset shall be available whenever the user profile service is active.
- Security: Only the authenticated account owner may reset their own profile preferences.
- Maintainability: Preference parameters and weights should be resettable through configurable profile services.
- Portability: The reset process should behave consistently across supported browsers.

Technology and Data Variations List:

- Preference data: Venue tags, category weights, historical interactions.
- Recommendation state: Personalized mode, neutral/default mode.

Frequency of Occurrence: Occasional.

Miscellaneous:

- In a future versions, users may be allowed to reset only selected preference categories instead of all preference data.
- The UI may optionally preview the effect of reset before confirmation.

Use Case Diagram UCD7: Preference Profile Reset

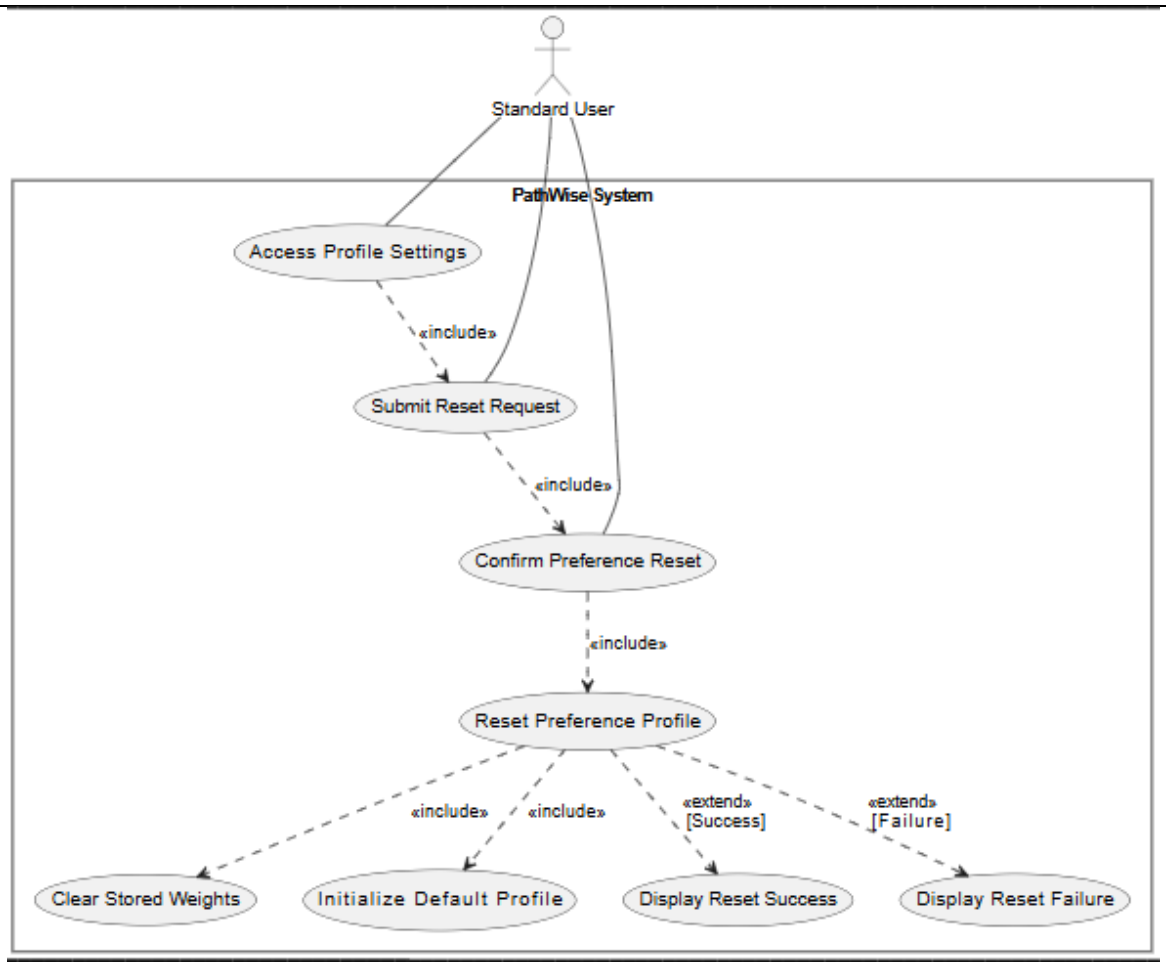


Figure 17: *Use Case Diagram #7*

4.1.8. USE CASE #8

Use Case UC8: Initial Preference Tag Selection

Scope: PathWise System.

Level: User Goal.

Primary Actors: Standard User.

Stakeholders and Interests:

- Standard User: Wants to select personal preference tags in order to receive accurate and personalized travel recommendations from the system.
- Recommendation Engine: Requires sufficient preference data to generate meaningful and relevant venue suggestions.

- System: Must ensure that a valid and complete preference profile is created before enabling access to recommendation functionalities.

Preconditions:

- PathWise system is operational and accessible.
- User has successfully registered an account.
- User logs into the system for the first time.

Success Guarantees:

- User selects at least 3 preference tags.
- Selected tags are successfully stored in the system database.
- User preference profile is initialized.
- Recommendation engine uses selected tags for generating personalized suggestions.
- User is redirected to the dashboard after completing the process.
- User is able to update preference tags later through profile settings.

Main Success Scenario:

1. System detects that the user is logging in for the first time.
2. System checks whether the user's preference profile has been initialized.
3. System redirects the user to the tag selection page.
4. System displays a predefined list of preference tags.
5. User selects at least 3 preference tags.,
6. User submits the selected tags.
7. System validates the selected tags.
8. System stores the selected tags in the database.
9. System initializes the user's preference profile.
10. System notifies the recommendation engine.
11. System displays a success message.
12. System redirects the user to the dashboard.

Extensions:

2b. User already has an initialized preference profile.

    1. System skips the tag selection process.
    2. System redirects the user directly to the dashboard.

5a. User selects fewer than the required minimum number of tags.

    1. System displays an error message indicating that at least 3 tags must be selected.
    2. System prevents the user from proceeding.

6a. User submits incomplete selection.

1. System displays an error message and highlights the issue.

8a. A database error occurs while storing preference tags.

1. System displays an error message to the user.
2. System prevents incomplete preference profile creation.

Special Requirements:

- Performance: The tag selection interface must respond within 2 seconds under normal conditions.
- Reliability: The system must ensure that no partial or inconsistent preference data is stored in case of failure.
- Availability: The tag selection page must be available whenever a first-time login occurs.
- Security: Only authenticated users are allowed to perform preference selection.
- Maintainability: The tag list must be easily extendable without requiring major changes in the system.
- Portability: The tag selection interface must be consistent across modern web browsers such as Chrome and Edge.

Technology and Data Variations List:

- Tag Categories: Travel style, budget preference, activity type, food preference, accommodation preference.
- Minimum Tag Requirement: At least 3 tags must be selected.

Frequency of Occurrence: Once per first login.

Miscellaneous:

- The system enforces mandatory completion of tag selection before allowing access to other system functionalities.
- The tag selection interface is designed to support future expansion of tag categories without affecting existing user data.

Use Case Diagram UCD8: Initial Preference Tag Selection

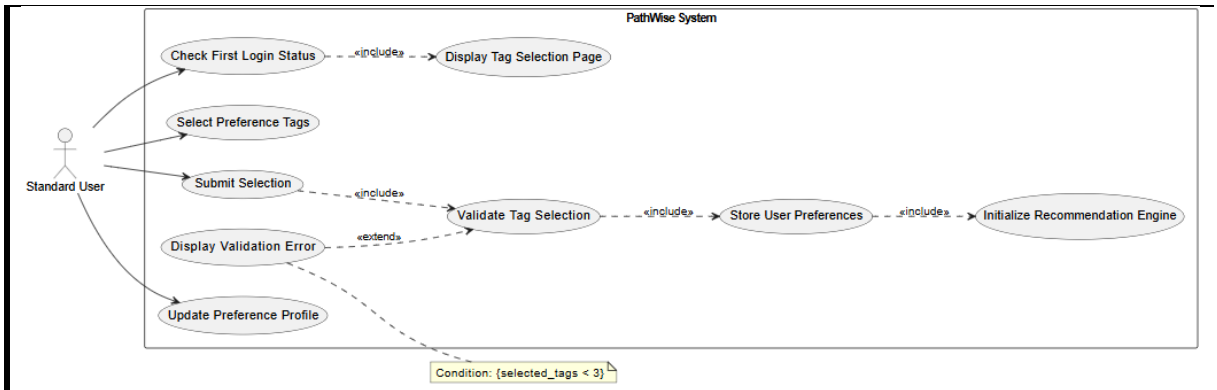


Figure 18: *Use Case Diagram #8*

4.2. CLASSES / OBJECTS

To ensure a highly cohesive and loosely coupled architecture, the system's core functionalities are divided into distinct high-level modules. This modular approach aligns with the separation of concerns principle, isolating business logic from data representation and user interface coordination.

1- Entity Module (Domain Models)

This module represents the core data structures and domain objects of the system.

- **User:** Manages demographic information, role assignments and account states.
- **Venue:** Represents the accommodation and dining locations, encapsulating attributes such as capacity, base price and availability.
- **Tag:** Defines the descriptive labels associated with users and venues, serving as the primary metric for the recommendation algorithm's weight calculations.
- **TripPlan (Itinerary):** Encapsulates the final generated travel plan, linking a specific user to a series of venues over a defined timeline and budget constraint

2- Algorithm Module (Business Model)

This module contains complex processing units isolated from standard data management.

- **RecommendationEngine:** The core processing class responsible for dynamically matching user constraints (budget, dates, traveler count etc.) with venue attributes and historical tag weights to generate optimal itinerary combinations.

3- Organization Module (Coordination & UI)

This module acts as the intermediary between the backend logic and the user interfaces.

- **PageController:** Manages user session routing, input handling, and the presentation of dynamic form (e.g. comprehensive planning templates)
- **NotificationManager:** Coordinates system-wide alerts, such as warning standard users about budget limits or notifying admins of pending venue approvals.

4- Security & Access Module

- **AuthManager:** Manages secure login operations, password hashing and strict role-based authorization to ensure users only access their designated dashboard.

5- Data Access Module

- **DataBaseManager/Repository:** Acts as an abstraction layer for persistent data storage, managing all database queries and transactions independently from the domain entities.

6- Log Module (Monitoring)

Tracing: Records system activities, algorithm decision logs and error states to facilitate debugging and support the project's technical decision-tracking requirements.

4.3. SEQUENCE DIAGRAMS

This subsection presents detailed sequence diagrams for the previously discussed use cases, illustrating the step-by-step interactions between the actors, the PathWise system, and the underlying database.

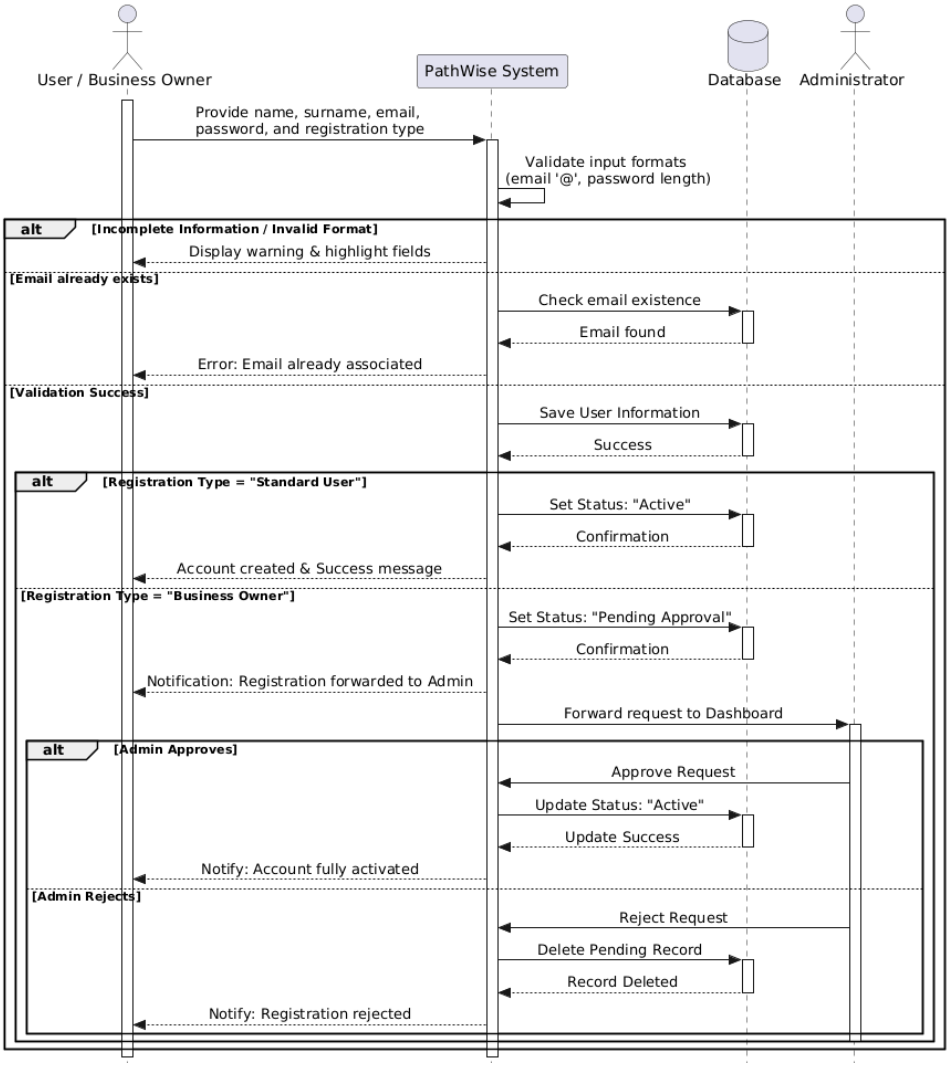


Figure 19: *Sequence Diagram for UC#1*

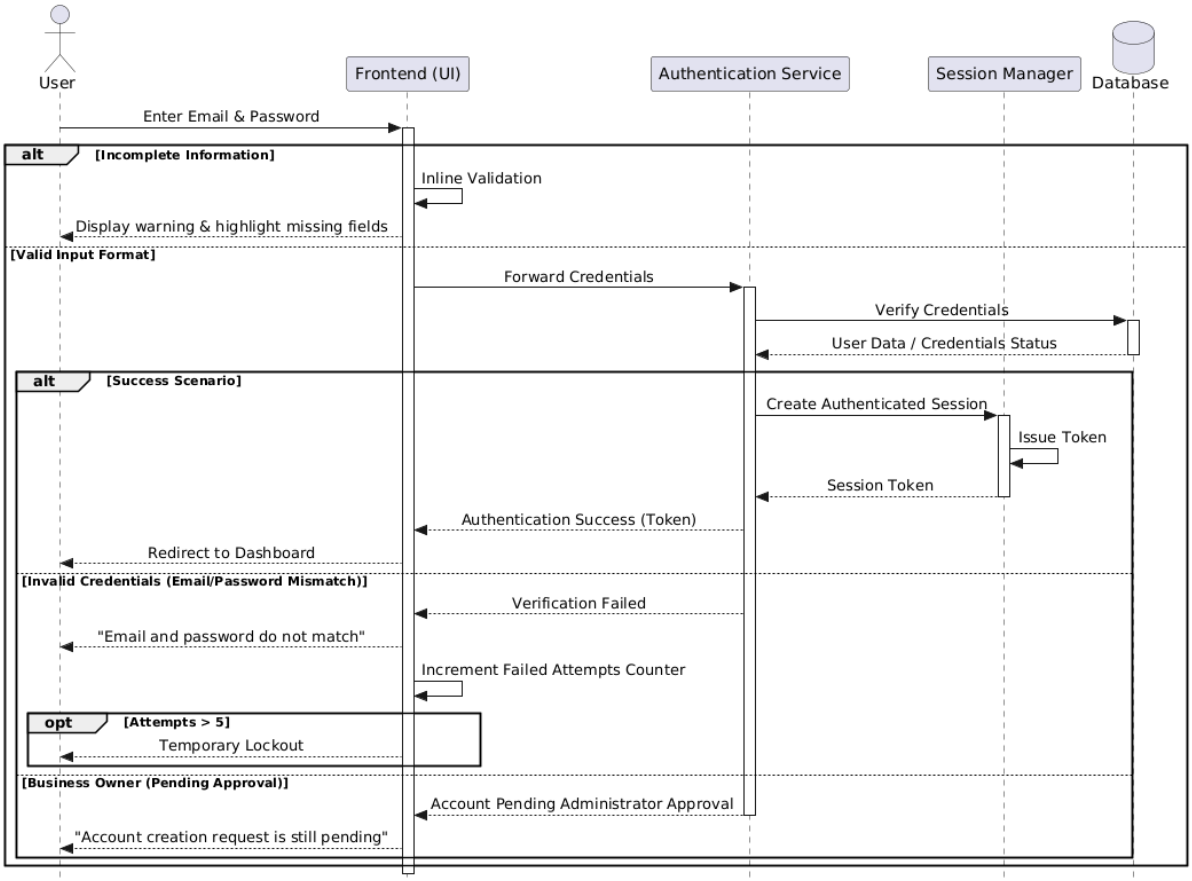


Figure 20: *Sequence Diagram for UC#2*

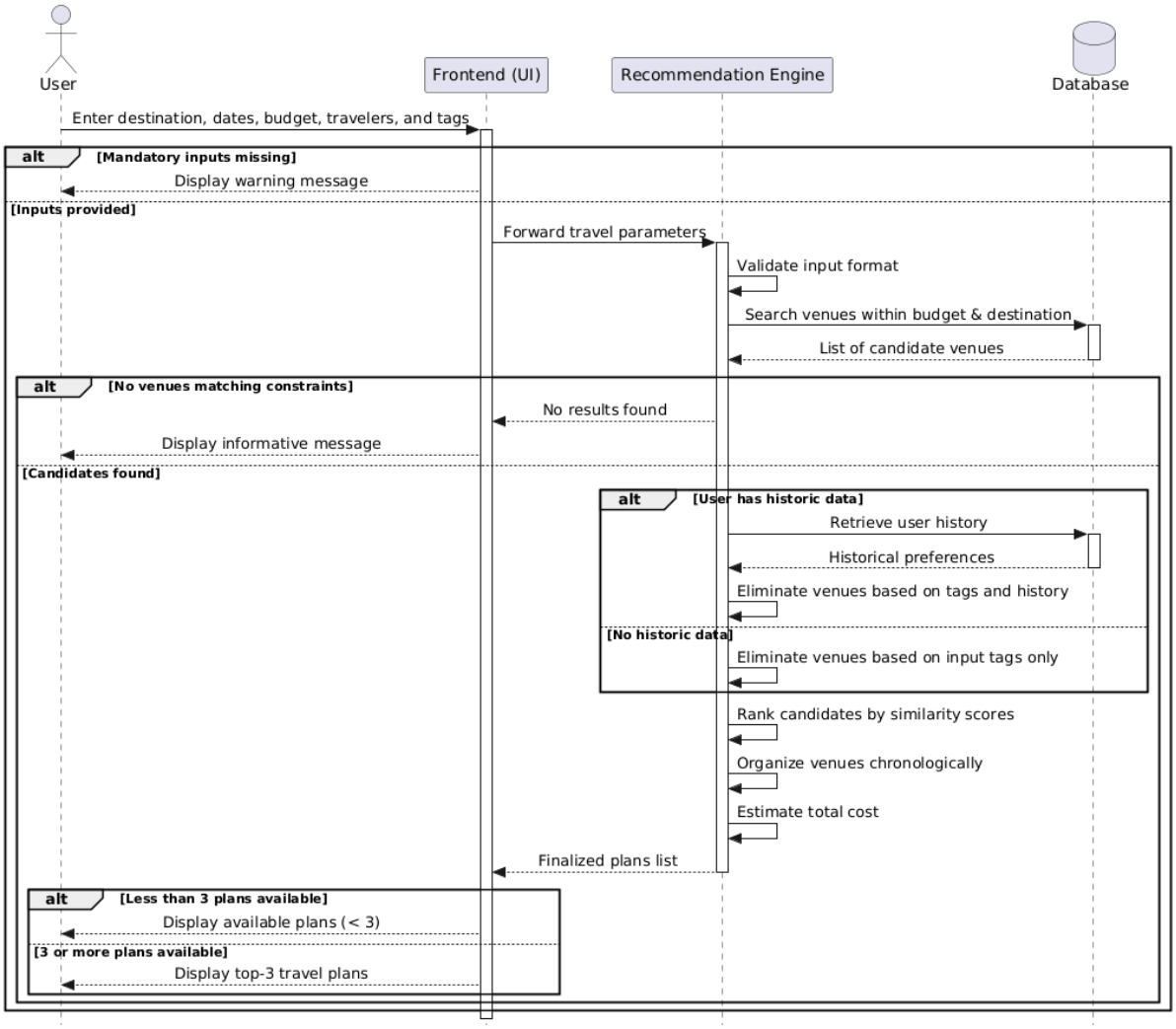


Figure 21: *Sequence Diagram for UC#3*

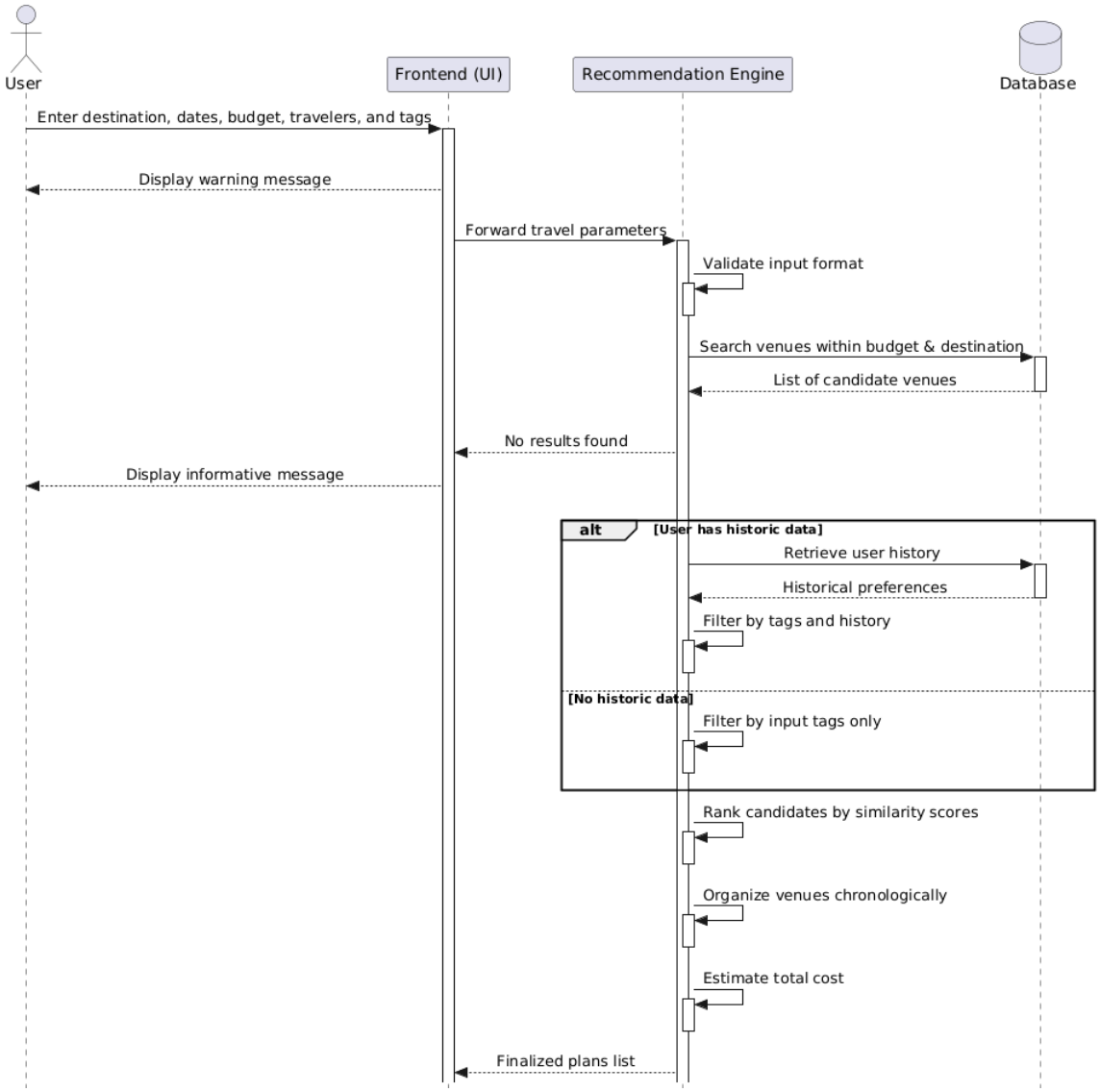


Figure 22: *Sequence Diagram for UC#4*

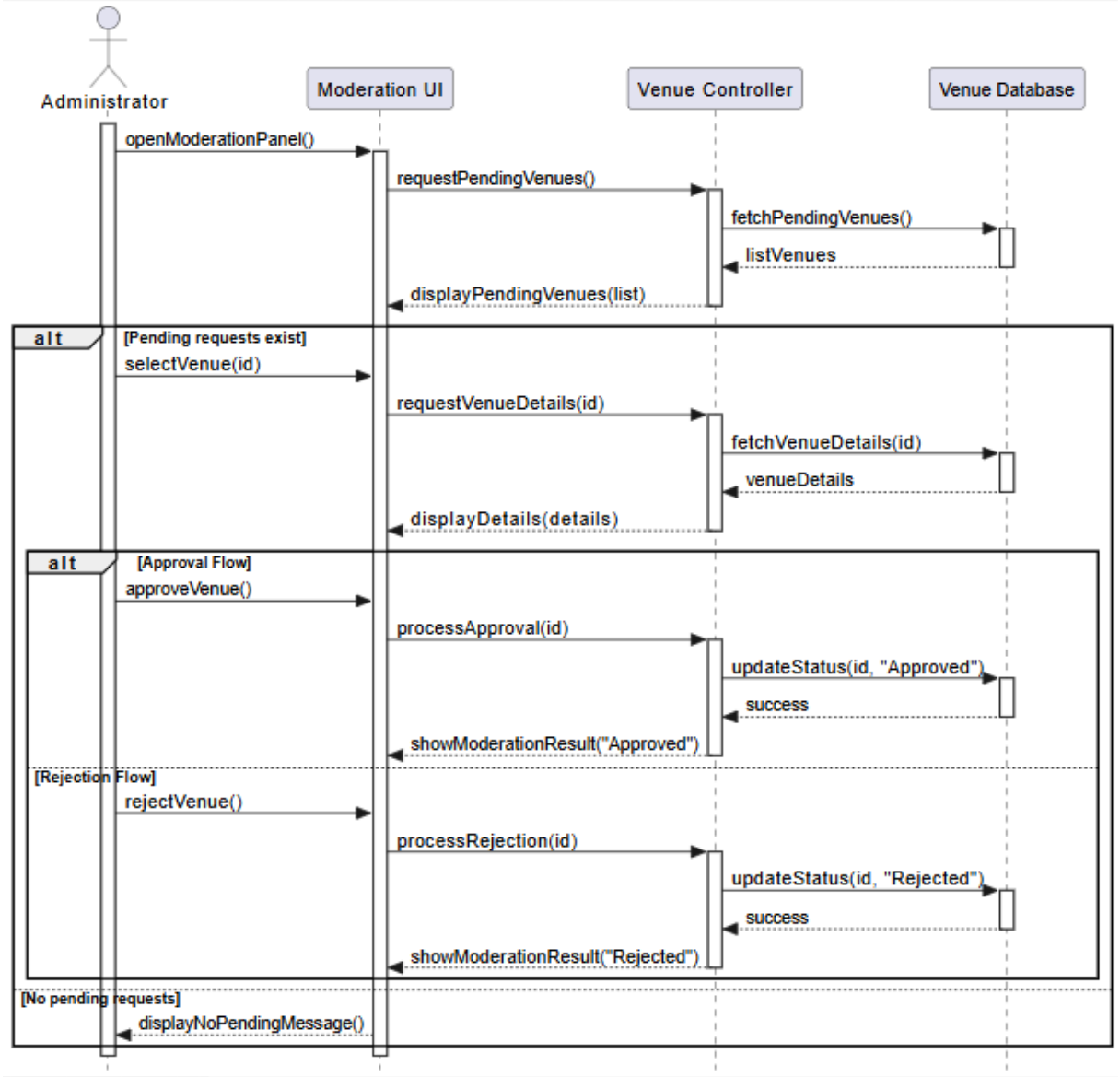


Figure 23: *Sequence Diagram for UC#5*

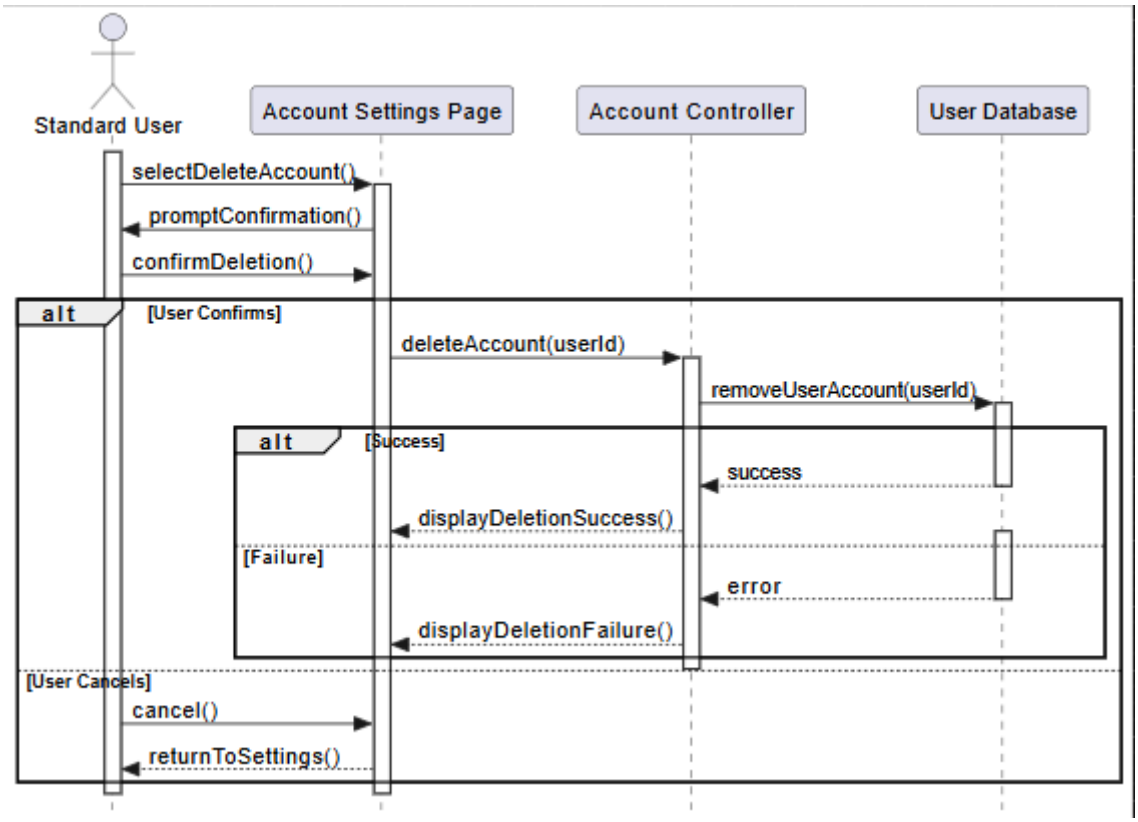


Figure 24: *Sequence Diagram for UC#6*

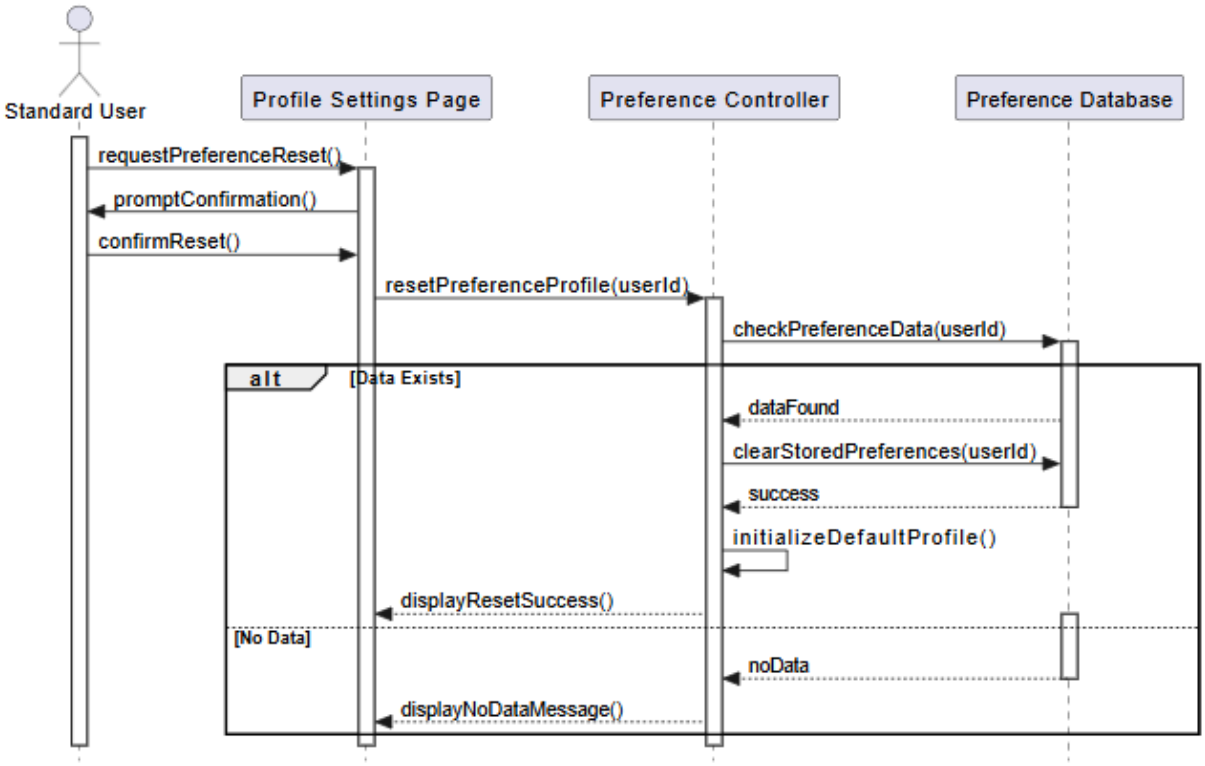


Figure 25: *Sequence Diagram for UC#7*

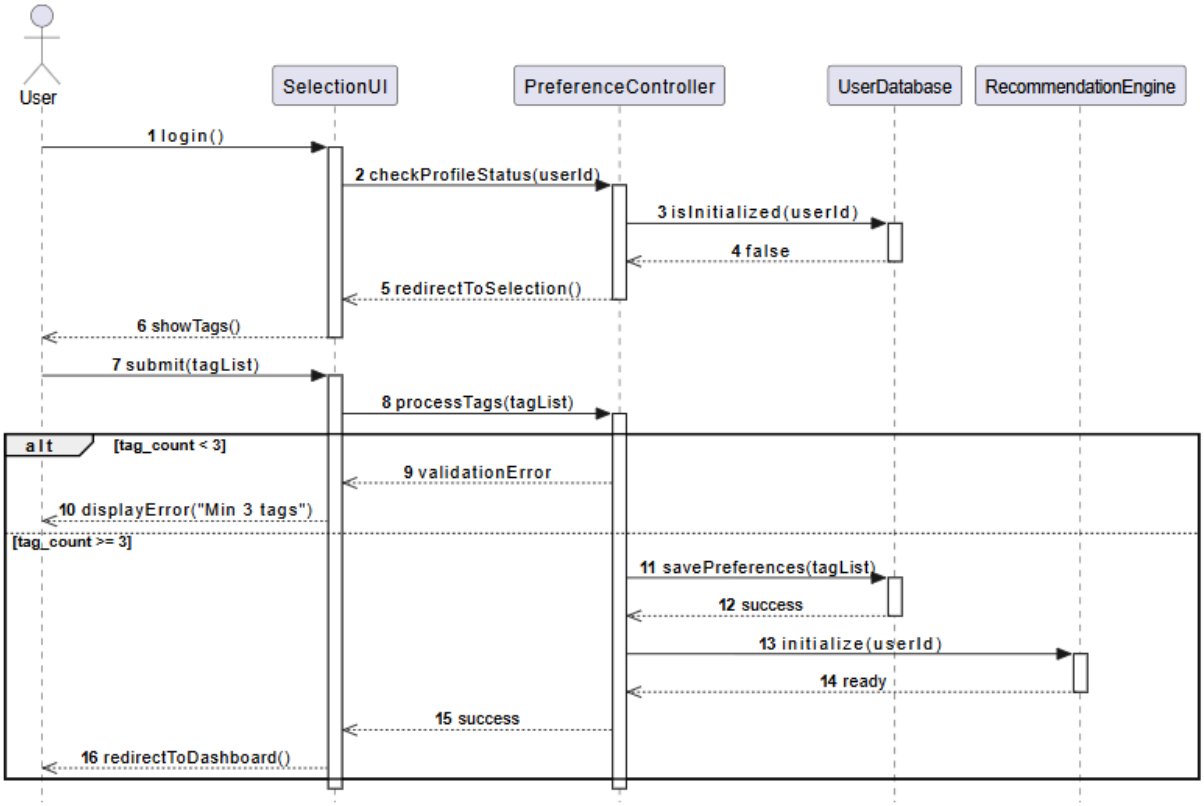


Figure 26: *Sequence Diagram for UC#8*

5. APPENDICES

5.1. APPENDIX A: DECISION LOG

This appendix includes the key decisions made during the analysis of the PathWise system. Each decision includes the selected approach, considered alternatives, justification, trade-offs, and potential risks. These decisions reflect the constraints and scope of the project and support the overall system design.

5.1.1. DECISION #1: TECHNOLOGY STACK SELECTION

5.1.1.1. DECISION MADE

The system is implemented using React for the frontend, ASP.NET Core Web Application Programming Interface for the backend, the C# programming language, a relational database management system, and Entity Framework Core as the object relational mapping tool.

5.1.1.2. ALTERNATIVES CONSIDERED

- Other frontend frameworks such as Angular and Vue.
- Other backend technologies such as Node.js.
- Direct database access using structured query language without an object-relational mapping tool.

5.1.1.3. RATIONALE

- Frontend (React): React provides a component-based architecture that allows reusable user interface elements and supports dynamic and interactive web applications.
- Backend (ASP.NET Core Web Application Programming Interface): This technology provides a structured and scalable way to develop backend services and supports the implementation of business logic and authentication mechanisms.
- Programming Language (C#): The C# programming language offers strong type safety and integrates effectively with the backend framework.

- Relational Database Management System (PostgreSQL): The system data, including users, venues, and recommendations, has a structured and relational nature, which is well supported by relational databases.
- Entity Framework Core: This tool simplifies communication between the application and the database by eliminating the need for manually writing structured query language statements and improves maintainability.

5.1.1.4. TRADE-OFFS

- Increased system complexity due to the use of multiple technologies.
- Additional learning effort required for development.
- Reduced control over database queries due to abstraction provided by the object-relational mapping tool.

5.1.1.5. RISKS OR TECHNICAL DEBTS

- Dependency on the selected technology stack.
- Possible performance issues if the object-relational mapping tool is not used efficiently.
- Future migration to different technologies may require significant refactoring effort.

5.1.2. DECISION #2: RECOMMENDATION APPROACH

5.1.2.1. DECISION MADE

The system uses a tag-based approach to generate personalized travel recommendations.

5.1.2.2. ALTERNATIVES CONSIDERED

Basic filtering without personalization (More advanced approaches such as artificial intelligence-based recommendation systems were not considered within the scope of this project.)

5.1.2.3. RATIONALE

- The tag-based approach allows personalization based on user-selected preferences
- It is suitable for the project scope and implementation timeline

- The logic is simple, interpretable, and easy to implement

5.1.2.4. TRADE-OFFS

- Limited adaptability compared to more advanced recommendation methods
- Recommendation quality depends heavily on the accuracy and completeness of tags

5.1.2.5. RISKS OR TECHNICAL DEBTS

- The recommendation system may not perform optimally with complex or large datasets
- Future improvements may require redesigning the recommendation logic

5.1.3. DECISION #3: RECOMMENDATION PLAN GENERATION

5.1.3.1. DECISION MADE

The system generates multiple alternative travel plans instead of a single recommendation.

5.1.3.2. ALTERNATIVES CONSIDERED

- Generating a single optimized travel plan

5.1.3.3. RATIONALE

- Providing multiple options allows users to compare and choose according to their preferences
- Improves user experience by offering flexibility

5.1.3.4. TRADE-OFFS

- Increased computational complexity
- More complex user interface design

5.1.3.5. RISKS OR TECHNICAL DEBTS

- Potential increase in response time
- Users may experience difficulty in decision-making if too many options are presented

5.1.4. DECISION #4: VENUE APPROVAL

5.1.4.1. DECISION MADE

Venue submissions by business owners must be approved by an administrator before becoming visible in the system

5.1.4.2. ALTERNATIVES CONSIDERED

- Direct publication of venues without moderation
- Automated validation based on predefined rules (Automated approaches were not implemented due to project simplicity and scope limitations.)

5.1.4.3. RATIONALE

- Ensures the accuracy and reliability of venue data
- Prevents invalid, incomplete, or misleading entries from being presented to users
- Maintains the overall quality and trustworthiness of the system

5.1.4.4. TRADE-OFFS

- Introduces delay between submission and visibility of venues
- Requires additional effort from administrators
- Reduces the level of system automation

5.1.4.5. RISKS OR TECHNICAL DEBTS

- Introduces delay between submission and visibility of venues
- Requires additional effort from administrators
- Reduces the level of system automation

5.1.5. DECISION #5: DEVELOPMENT ENVIRONMENT

5.1.5.1. DECISION MADE

The system operates in a local development environment and is not deployed to a production server.

5.1.5.2. ALTERNATIVES CONSIDERED

- Deployment to cloud-based platforms such as remote servers (These alternatives were not explored due to project scope and infrastructure limitations.)

5.1.5.3. RATIONALE

- Simplifies development, testing, and debugging processes
- Eliminates the need for infrastructure and deployment management
- Suitable for an academic project and demonstration purposes

5.1.5.4. TRADE-OFFS

- Limited accessibility, as the system is only available locally
- Lack of testing under real-world conditions such as network latency and high user load

5.1.5.5. RISKS OR TECHNICAL DEBTS

- System behavior may differ in a real production environment
- Future deployment may require additional configuration, security improvements, and performance optimization
- Integration with external systems may require further adjustments

5.1.6. DECISION #6: DATA STORAGE

5.1.6.1. DECISION MADE

A relational database management system is used for storing and managing system data.

5.1.6.2. ALTERNATIVES CONSIDERED

- Non-relational database systems (These alternatives were not explored in detail due to the structured nature of system data.)

5.1.6.3. RATIONALE

- The system contains structured data with clear relationships between entities such as users, venues, and recommendations
- Supports data integrity through constraints and relational structure
- Compatible with the object-relational mapping approach used in the system

5.1.6.4. TRADE-OFFS

- Less flexibility for handling unstructured data
- Schema modifications may require additional effort and database migration

5.1.6.5. RISKS OR TECHNICAL DEBTS

- Potential scalability limitations as the dataset grows
- Complex queries may impact performance if not optimized
- Future improvements may require query optimization or migration strategies

5.1.7. DECISION #7: RATING-BASED FEEDBACK MECHANISM

5.1.7.1. DECISION MADE

The system includes a simple rating-based feedback mechanism for evaluating recommendations.

5.1.7.2. ALTERNATIVES CONSIDERED

- No feedback mechanism
- Detailed review or comment system (More advanced feedback systems were not included due to scope limitations.)

5.1.7.3. RATIONALE

- Provides an easy and quick way for users to give feedback
- Helps evaluate the usefulness of generated recommendations

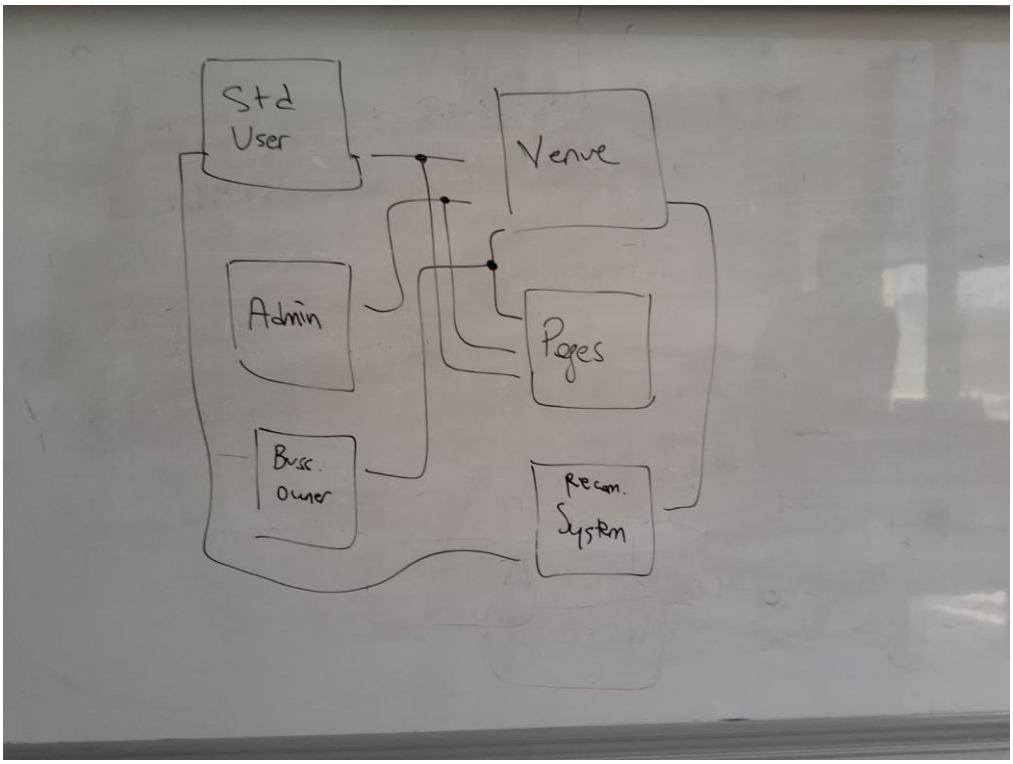
5.1.7.4. TRADE-OFFS

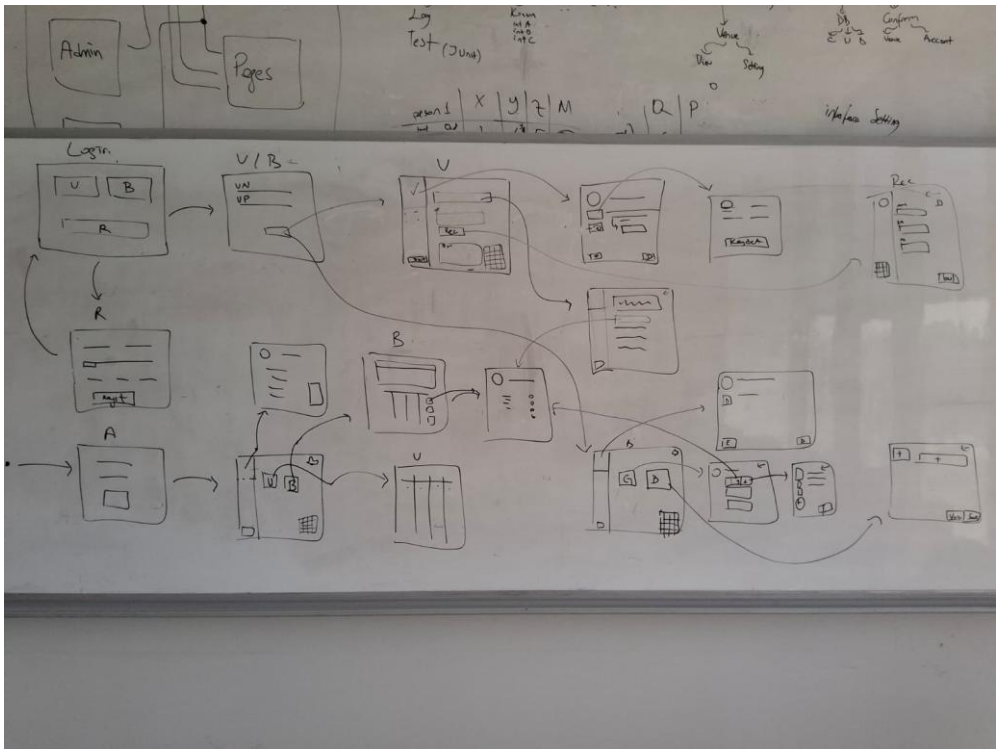
- Limited level of detail in user feedback
- Does not capture qualitative user opinions

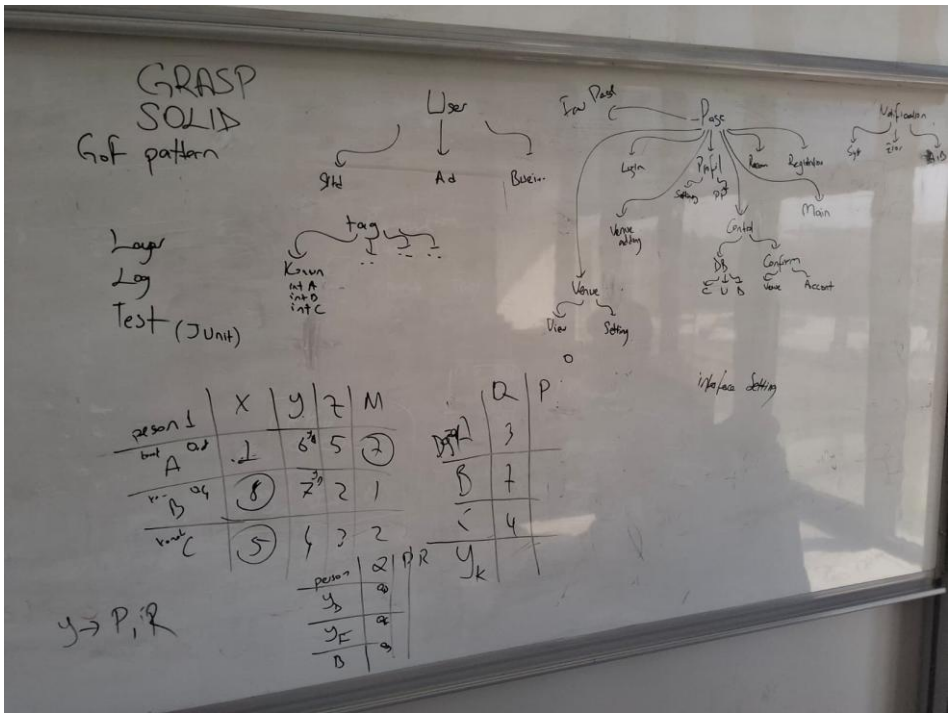
5.1.7.5. RISKS OR TECHNICAL DEBTS

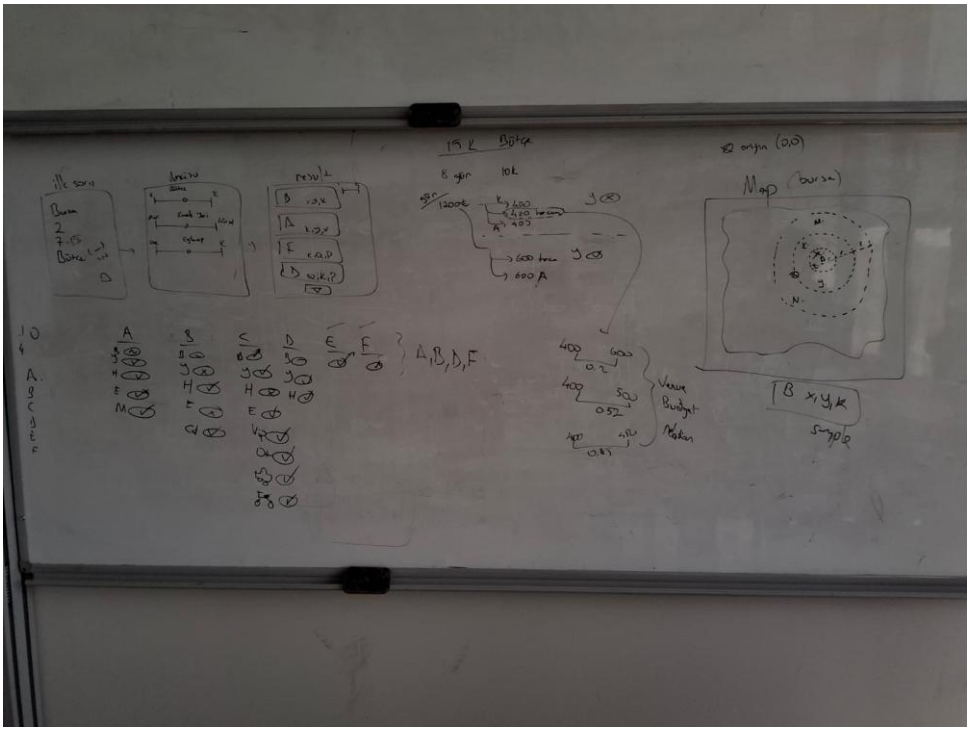
- Insufficient feedback data for improving recommendation logic
- May require future extension to more advanced feedback mechanisms

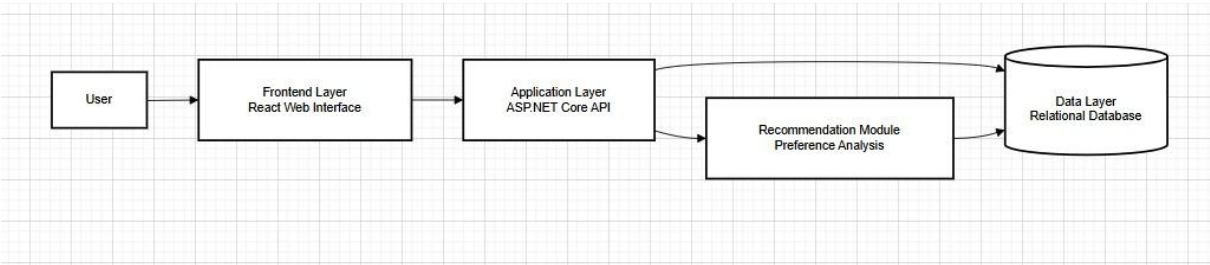
5.2. APPENDIX B: TEAM MEETINGS






GRASP
SOLID
GoF pattern
User
Ad
Main
Layer
Log
Test (JUnit)
tag
Login
Registration
Main
Venue
Setting
Interface Setting


Map
origin (0,0)


User
Frontend Layer
React Web Interface
Application Layer
ASP.NET Core API
Recommendation Module
Preference Analysis
Data Layer
Relational Database